Digital Architecture Framework



Digital Architecture

Framework

DAF v. 7.0.0 Beta 6

Rev. 18

Nov 2026

Contents

About Digital Architecture Framework (DAF)....3
License....6
Project....7
Installation....7
What's new in DAF 7.0....9
Concepts....13
From Reality to Abstraction....13
From the Model to a View....13
DAF Metamodel....16
Example Model Instance....17
Layers....19
Enterprise Architecture....19
Solution Architecture....19
Design Architecture....20
Domains....20
Business....20
Data....20
Application....20
Technology....21
Extension Domains....21
Viewpoints and Views....22
DAF Concepts Organized in Viewpoints....24
Views and work products....25
Relationships....26
Relationship Strategy....27
Logic in DAF....28
Process logic....28
Application logic....28
Business rules....28
Modeling controllers and rules....29
Rules found in existing code....29
Detailed Documentation....29
DAF Concepts Overview....30
DAF Relationship Types....47
DAF Relationships Quick link....58
Visual Characteristics of some DAF stereotypes....66
Appendices....68
Stereotypes and Tagged Values....68
Enumerations....95
Model Checks....99
DAF COLORS....105
RGB Colors....105
HEX Colors....105

1 About Digital Architecture Framework (DAF)

The Digital Architecture Framework (DAF), formerly known as "Delta Architectural Framework," is a set of modeling tools for Digital Engineering aiming to enable collaboration between roles such as Enterprise Architects, Solution Architects, and Business Analysts in designing the enterprise by creating high-quality models that can be transformed into artifacts such as code and documentation.

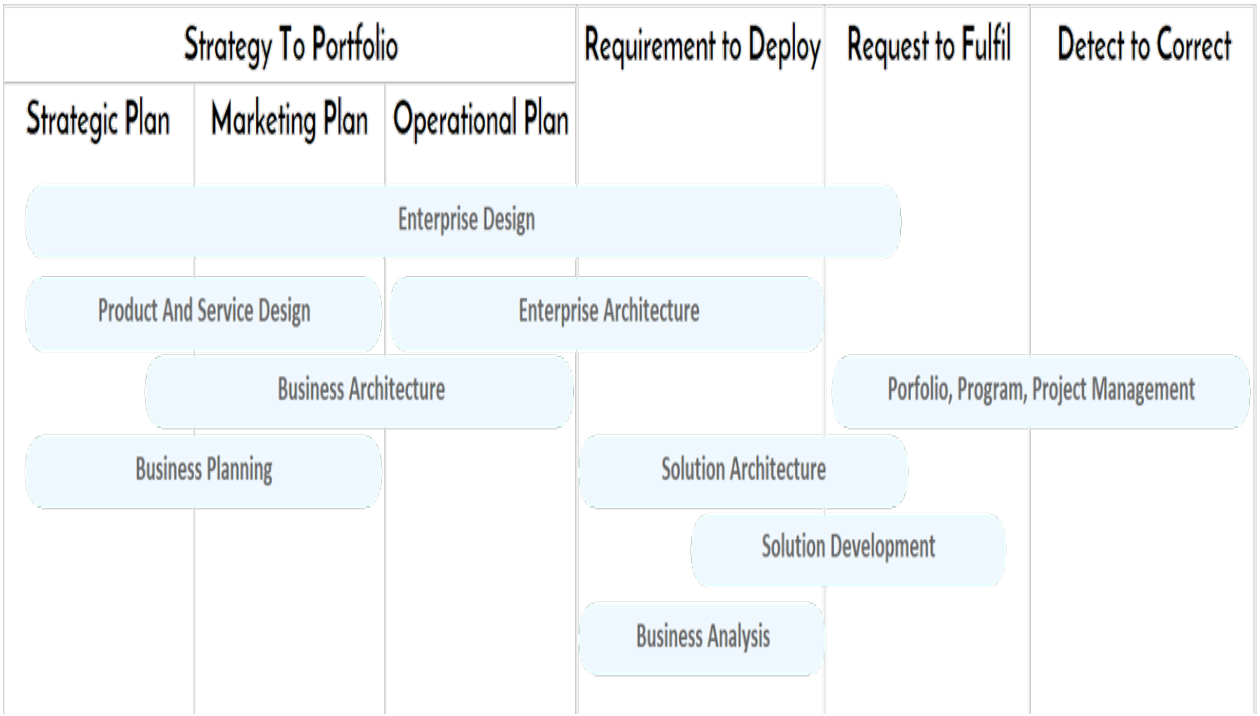


Figure 1: Value Streams covered by DAF

The diagram shows the classic gaps between strategy, design, and execution. DAF may accelerate the delivery of projects by supporting collaboration of different roles and by creating traceability between strategy and execution. In fact, it provides the metamodel to close these gaps, ensuring that strategic plans are directly reflected in architectures, architectures are transformed into solutions, and solutions generate tangible deliverables with traceability back to strategy.

The DAF meta-model is based on industry standards such as UML, ArchiMate, TOGAF, RUP, SPEM and best practices for requirement management, and offers a baseline of 78 concepts, 118 relationships and 44 enumerations that can be customized to meet specific needs. While DAF can be used on its own, it is not intended to represent a new standard but rather serves as an accelerator that allows for quick tailoring of existing standards.

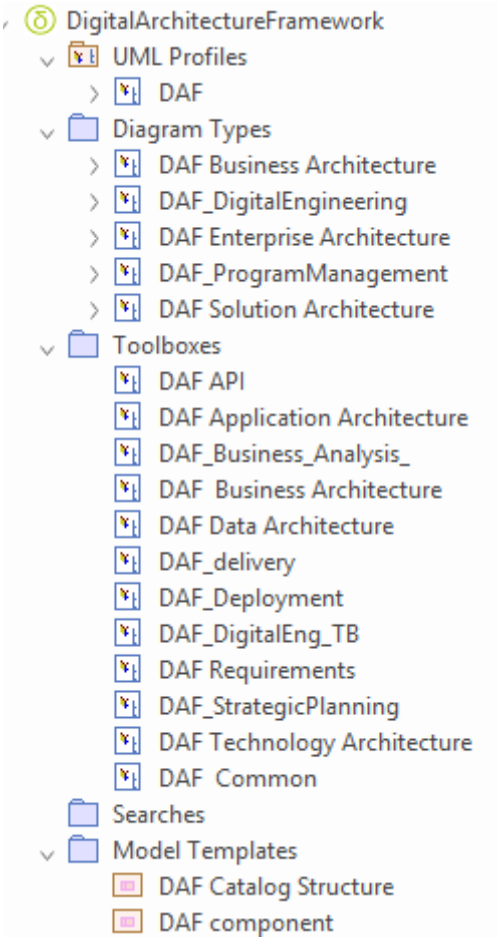


Figure 2: Content of the DAF MDG

DAF is currently implemented as MDG technology in Sparx Enterprise Architect, including:

- UML profile
- A set of customized icons
- Shape scripts
- multiple Toolboxes, organized by roles
- Custom diagrams, organized by roles
- Model patterns, accelerating the creation of DAF diagrams.
- Model Template with the structure of DAF Catalogs
- Quick linker
- A set of 100+ diagram examples showing how to use DAF to cover Enterprise Architecture, Solution Architecture, Design Architecture and Business Analysis. Those examples include all the diagrams in the TOGAF 9.1 specification.
- A structure based on TOGAF to host an enterprise repository
- A set of scripts, performing Model-to-Model (M2M) transformations
- A set of rules for model validation implemented in the EA Validator

- A semantic model of the metamodel (RDF/OWL with SHACL shapes), generated from the Sparx model

2 License

- DAF is Open Source, released under the Creative Commons
- 

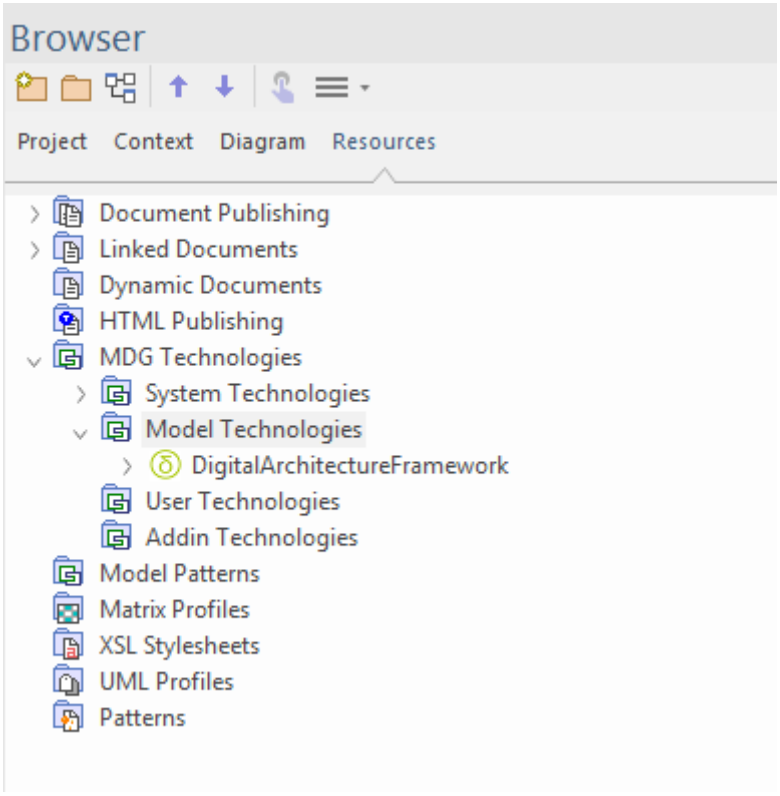
- Digital Architecture Framework © 2026 by Giu Platania is licensed under Creative Commons Attribution 4.0 International
- This license requires that reusers give credit to the creator. It allows reusers to distribute, remix, adapt, and build upon the material in any medium or format, even for commercial purposes.

3 Project

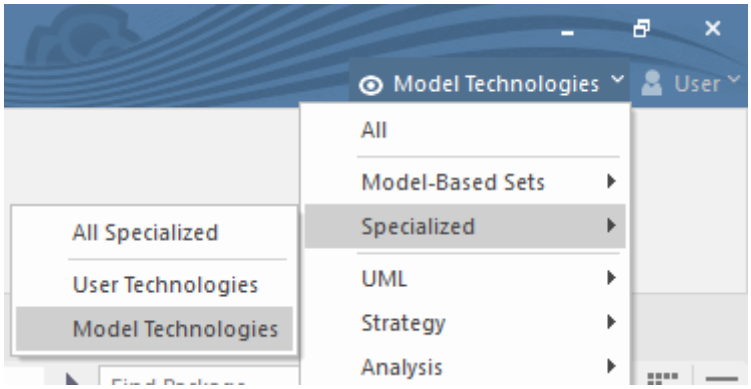
DAF is maintained in GitHub: Digital Architecture Framework GitHub repository

4 Installation

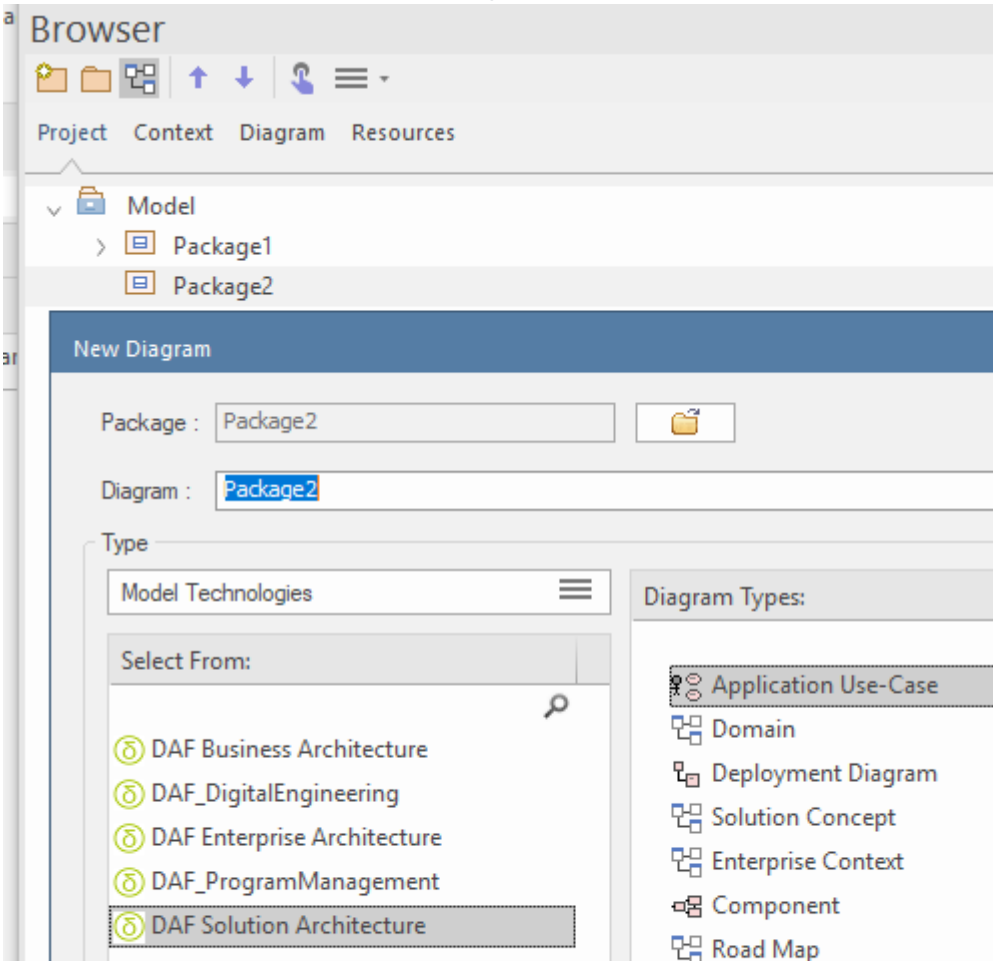
1. **Obtain the DAF MDG file**: Ensure you have the XML file defining the DAF MDG Technology from the repository Digital Architecture Framework releases. This file contains the definitions for stereotypes, toolboxes, profiles, and diagram types.
2. Save the file to an accessible directory on your computer.
3. Launch **Sparx Enterprise Architect**.
4. Open or create a project where you want to use the DAF MDG Technology.



5. In the browser, open the tab Resources and navigate to **MDG technologies> Model Technologies**, right click and select **Import Technology**.
6. Locate and select the daf.xml file
7. Select **import to Model**
8. Click **OK** to load the MDG Technology into Sparx EA.
9. **Optional: Enable the DAF MDG Technology**
    - Go to **Specialize > Technologies > Manage Technologies**.
    - In the list of available MDG Technologies, find "DAF" and check the box to enable it.
    - Click **Close** to save your settings.
10. **Suggested:** set your perspective focus on DAF



11. Test a DAF diagram
    - Create a new diagram
        - Select one DAF Diagram from the options



5 What's new in DAF 7.0

DAF 7.0 is a major release: it changes datatypes, renames relationships and adds concepts, so models created with DAF 6.x must be migrated (see Migrating models at the end of this chapter). The current version is DAF 7.0.0 Beta 6. The changes are tracked in GitHub issue #4, found while validating a complete test model against the DAF 6.5.1 semantic model and checker rules, and, for the Beta, in issue #5.

In numbers: 78 concepts, 118 relationship types and 44 enumerations. Every relationship end has a multiplicity, and every property has a type, a multiplicity and a description.

5.1 No invented values

An absent value now stays unknown. DAF 6.x forced modelers to fill in many properties and supplied placeholder defaults; in analysis and simulation, those defaults read as confident but wrong data.

- Money, volume and duration properties use the new Quantity value type and are optional, for example dRole.FTEs, dOrganizationUnit.HeadCount, dInitiative.EstimatedTotalCost, dProduct.Price and dMeasurementIndicator.CurrentLevel.
- Only 31 properties are required; all others are 0..1 or 0..*. Optional properties have no default value, so a model, or an LLM filling it, never inherits a value nobody assessed.
- Every property has a description that says what it holds, its format or unit with an example, and what an empty value means.
- RetireDate is a Date, 0..1, on all five concepts that have it. All JSON Schema facets are optional.
- dInitiative.InitiativeDuration is a Duration, and dInitiative.Benefit is removed. dRisk.likelihood is an optional Int32 from 1 to 100.

5.2 New concepts and value types

Concept	Purpose
Quantity (dQuantityValue)	A value with a unit: amount (Decimal), unit (ISO 4217 currency or UCUM code), per, basis, asOf and kind (Actual, Estimate, Assumption).
Cost Period (dCostPeriod)	All spend in DAF: the cost of an initiative, application, role, capability, product or resource in one period, or the budget cap of a program. costNature says Capex or Opex; costFunction says which income-statement line an operating cost lands on (cost of revenue, SG&A, R&D).
Revenue Period (dRevenuePeriod)	Revenue earned by a product (dProductHasRevenuePeriod) or a contract (dContractHasRevenuePeriod) in one period.
Financial Period (dFinancialPeriod)	An organisation unit's reported or planned results in one period: revenue, cost of revenue, operating and revenue-linked expense, depreciation, interest, investment income, invested assets, income tax and capital expenditure. Linked by dOrganizationUnitReportsFinancialPeriod.
Contract (dContract)	Commercial terms: pricingBasis (dPricingBasis), billableUnit, tariff, effectiveFrom and effectiveTo, and renewal. Linked to products and actors.
Rule (dRule)	A business rule: a precise condition or constraint, enforced at one point in an application. It is a stereotyped operation, with visibility package, of the dController that enforces it; the controller is the rule set. RuleKind (dRuleKind) says Production, Integrity, Derivation or Reaction, Statement holds the rule in plain language, and SourceEvidence and Certainty (dRuleCertainty) say whether it was modeled or found in existing code. See Logic in DAF.

M3 gains the primitive types Decimal (exported as xsd:decimal) and Duration (ISO 8601, e.g. P1M, exported as xsd:duration), plus the new enumerations dCostNature, dCostFunction, dCostScope, dDepreciationMethod, dEventType, dEventNature, dPricingBasis, dQuantityKind, dRuleKind and dRuleCertainty.

5.3 Costs, revenue and financial periods

Cost and financial periods are elements linked by relationships, not property values, so each period can be traced, referenced and compared. A financial period includes the cost and revenue periods that land in its organisation unit's books (dFinancialPeriodIncludesCostPeriod, dFinancialPeriodIncludesRevenuePeriod), so every figure can be traced to its source and is never added twice.

- A financial period holds the income-statement lines (revenue, cost of revenue, operating expense, depreciation, interest, tax) and the cash line capitalExpenditure. Subtotals such as gross margin, EBITDA, EBIT, net income and free cash flow are derived, never stored.
- Included cost periods are checked by line: CostOfRevenue against costOfRevenue, SellingGeneralAdmin and ResearchDevelopment against operatingExpense, Capex against capitalExpenditure.
- Cost periods belong to exactly one owner: dInitiativeHasCostPeriod, dApplicationComponentHasCostPeriod, dRoleHasCostPeriod, dCapabilityHasCostPeriod, dProductHasCostPeriod or dResourceHasCostPeriod. A program's budget periods (dProgramHasBudgetPeriod) are caps and are never included.
- dApplicationComponent.Cost and dCapability.Cost are removed. dRole.CostRate holds the cost per FTE, a rate rather than spend, and a contract's tariff can have several price components.

Assets and depreciation. Resources and applications can be capitalised assets, with inServiceDate, usefulLife and depreciationMethod. Their Capex is recorded as cost periods; the depreciation schedule is derived from it and is not stored.

5.4 Events, KPIs and goals

- dEvent is both a trigger and an effect, with eventType, nature, frequency, likelihood, startDate, endDate, duration and owner (a dRole), all optional.
- Four relationships form the flow between events and processes: dEventTriggersProcess, dProcessResultsInEvent, dProcessFlowsToProcess and dEventLeadsToEvent.
- KPIs can improve or worsen other KPIs, a goal can target KPIs (dGoalTargetsdMeasurementIndicator), and dActor.Count holds a population.
- A goal's Value_amount is the baseline and Value_Goal the change from it; the target is their sum. The deadline is the latest finishDate of the goal's objectives, and an objective can carry its own Value_Goal milestone.

5.5 Strategy and change

- Initiatives say what they change: dInitiativeDeploysApplication, dInitiativeRetiresApplication, dInitiativeChangesProcess and dInitiativeChangesLogicalComponent. dInitiative.actualStartDate and actualFinishDate record what happened against the plan.
- dRoleStaffsService (the role does the service's work) is separate from dRoleConsumesService (the role uses its output). dRoleMaintainsApplication and dProcessControlsProcess are new.

- Capability maturity is optional (absent means not assessed), and dIncrement members carry their order 0 to 5. CapabilityLevel is renamed HierarchyLevel. A capability's own cost periods hold only direct cost; its full cost is derived from its linked roles and applications, so CostScope is deprecated.
- Types are fixed: dRisk.Severity is an Int32, dRole.DegreeOfUtilisation a Quantity in %, yes/no fields are Boolean, and JSON size limits are Int32.

5.6 Logic and business rules

- New concept dRule (Rule): a business rule, modeled as a stereotyped operation with visibility package of a dController. The controller is the rule set. See Logic in DAF.
- Its tagged values are RuleKind (Production, Integrity, Derivation or Reaction, the OMG rule categories), Statement, SourceEvidence and Certainty (Explicit or Inferred). The new enumerations are dRuleKind and dRuleCertainty.
- Rule is on the Model page of the Digital Engineering toolbox, after Controller.

5.7 Relationships and names

Every relationship end has a multiplicity of the form n, n..m or n..*, and ends that blocked realistic models are opened. Misspelled names are corrected, and some properties became relationships:

DAF 6.x / 7.0 Alpha	DAF 7.0
dApplicationComponentRealizesogicalComponent	dApplicationComponentRealizesLogicalComponent
dApplicationAgregatesFunction	dApplicationAggregatesFunction
dControllerControlIsModel	dControllerControlsModel
dApplicationIRunsInNode	dApplicationRunsInNode
dRiskIRelatedToRisk	dRiskIsRelatedToRisk
dBusinessAreadHasCapability	dBusinessAreaHasCapability
dRole.#FTEs	dRole.FTEs
dCapability.CapabilityLevel	dCapability.HierarchyLevel
dInitiative.costProfile (property)	dInitiativeHasCostPeriod (relationship)

DAF 6.x / 7.0 Alpha	DAF 7.0
dProgram.budgetProfile (property)	dProgramHasBudgetPeriod (relationship)
dOrganizationUnit.financials (property)	dOrganizationUnitReportsFinancialPeriod (relationship)
dApplicationComponent.Cost, dCapability.Cost	cost periods (dApplicationComponentHasCostPeriod, dCapabilityHasCostPeriod)
dRole.Cost	dRole.CostRate (a rate) or cost periods (dRoleHasCostPeriod)

The properties Likehood, MounthPath and plattform are now likelihood, MountPath and platform. Duplicate enumerations are merged, MimeTypes holds generic IANA media types, and ArchitectureType adds s390x and ppc64le.

5.8 Model checks

- Check 009 follows the metamodel direction. Check 002 reports elements still in status Proposed.
- Check 013 accepts components whose status is not set.
- Rules that name a renamed relationship accept both the 6.x and the 7.0 name until the model is migrated.

5.9 MDG technology

- UML profile and MDG technology (technology/DAF.xml) regenerated as 7.0.0 Beta 6.
- Every stereotype appears on a toolbox page, and the new stereotypes have icons and shape scripts.
- The Strategic Planning toolbox has eight pages: Strategy, Portfolio, Finance and Operating Model, each with its relationships. Finance holds Org Unit, Financial Period, Cost Period, Revenue Period, Product, Contract and Role.
- The QuickLink carries the 7.0 rules.
- The semantic model technology/rdf/DAF-semantic-model.ttl (RDF/OWL with SHACL shapes) is generated from the Sparx model; the tables in this manual are generated from it.

5.10 Migrating models

Import the 7.0 MDG technology into the model, then migrate the elements and connectors. MIGRATION.md, at the root of the repository, records every change a model must go through and the rule for converting each one: Part 1 from DAF 6.5.1 to 7.0 Alpha, Part 2 from 7.0 Alpha to 7.0.0 Beta 3, and Part 3 from Beta 3 to Beta 4.

Nothing is guessed: a value is converted only where the conversion is certain; anything else stays unknown and is reported, with the element, the property, the original value and the reason, for a person to review. For example, a number without a unit becomes a Quantity only when its unit is known, a value equal to a removed 6.x default is flagged, and a cost becomes a cost period only when its period is known. Renamed properties and relationships are mapped to their 7.0 names.

6 Concepts

6.1 From Reality to Abstraction

DAF is a metamodel (M2) that describes the elements of a Model (M1) that is, in turn, an abstraction of a real system (M0).

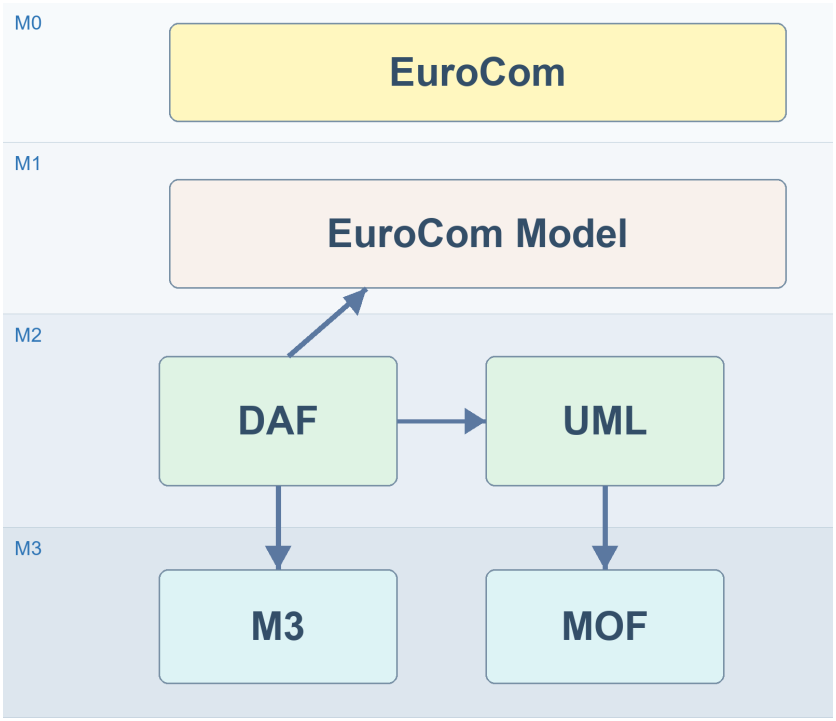


Figure 2: The hierarchy in this diagram follows the MDA principle of abstraction, where each level (M0 to M3) adds structure and abstraction to the level below, ensuring a consistent and scalable modeling approach.

- **M0: EuroCom:** The M0 layer represents the real-world organization or domain of interest, here as "EuroCom." This is the concrete instance of the domain that the models aim to represent, analyze, and improve. It consists of actual data and entities in the operational environment.
- **M1: EuroCom Model**: The M1 layer contains the model of the M0 domain. In this case, the "EuroCom Model" represents an abstraction of the EuroCom organization, describing its structure, behavior, or other aspects at a higher abstraction level. The M1 model acts as a blueprint that specifies how the M0 system is represented.
- **M2: DAF and UML**: The M2 layer defines the metamodels that describe the structure and semantics of the models in the M1 layer The arrow from DAF to UML indicates influence, highlighting how DAF integrates concepts from UML.
- **M3: M3 and MOF**: At this level, the fundamental rules and concepts for defining modeling languages are established. The M3 layer represents the meta-metamodels that describe the foundation for defining metamodels like UML or DAF. M3 is a custom meta-metamodel that can describe DAF or other metamodels. M3 is used instead of MOF (Meta-Object Facility): A standard meta-metamodel defined by the OMG to provide a formal foundation for modeling languages like UML.

6.2 From the Model to a View

There is a formal way of separating the idea of “one model” from the way we look at it, develop it, and communicate it.

Metamodel	Model	Layers	Viewpoint	View
A model documenting the elements used in a specific area of interest and their relationships.	A set of entities that describes the area of interest and its relationships.	A level of abstraction within the considered model. There are three layers in the DAF model.	A way of looking at the underlying model for a particular type of observer. There are 12 viewpoints in the DAF model.	What is seen from a viewpoint. Diagrams, matrices, and catalogs are types of views.

The Digital Engineering approach aims to enable the separation of concerns across different models, metamodels and viewpoints, allowing for a more flexible, maintainable and understandable representation of the system, making it possible for different stakeholders to focus on the aspects of the system that are relevant to them.

A "**metamodel**" is a model … of a model, which defines the structure and constraints of the models that conform to it. It describes the concepts, properties, and relationships that a model can contain, as well as the constraints and rules that apply to them. Metamodels are often expressed in formal languages, such as MOF or eMOF.

The concept of a "**model**" refers to a representation of a system or part of it in a specific format that adheres to the metamodel (e.g. UML or ArchiMate). These models can be used to define the architecture of a system. The ISO/IEC/IEEE 42010 standard defines "*architecture*" as "*the fundamental organization of a system, embodied in its components, their relationships to each other and the environment, and the principles guiding its design and evolution*."

.

A "**Model Layer**" is a set of models that conform to a specific metamodel and are used to represent a specific aspect of the system being modeled. For example, one model layer could be used to represent the logical architecture of the system, while another could be used to represent the physical deployment of the system.

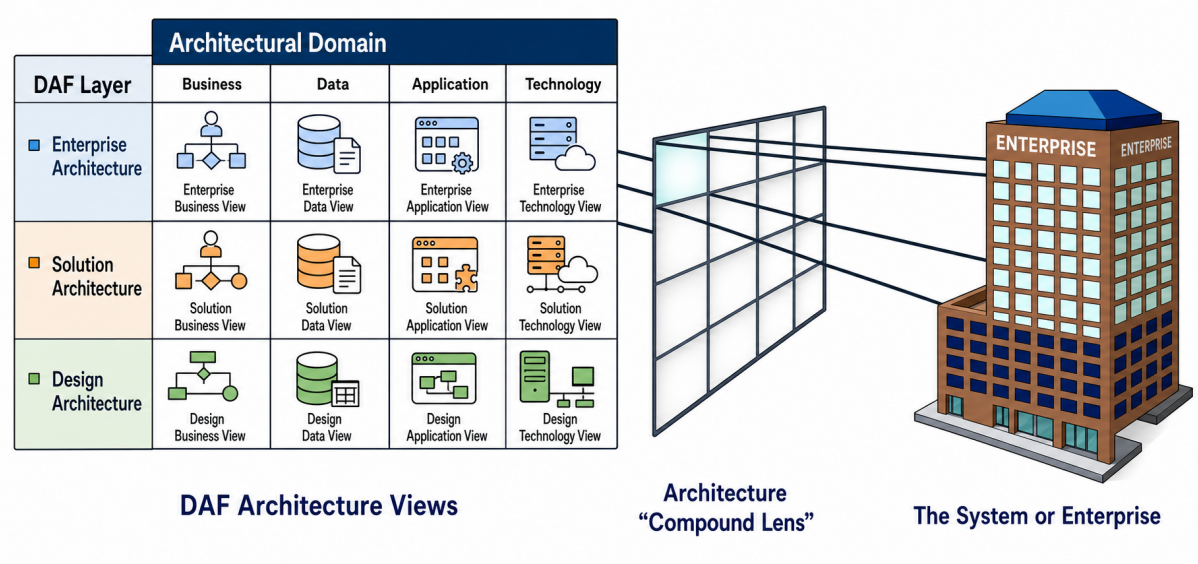


Figure 3: Relation between IT view and Enterprise

A "**Viewpoint**" is a set of views that have a common purpose or perspective. It can be related to specific domain, such as security or performance, and defines a specific set of views and model elements to be used to capture the relevant information from the system.

A "**View**" is a perspective or a specific part of a model that is relevant to a particular audience, such as stakeholders or developers. Each view provides a specific representation of the system, highlighting different aspects of the system and omitting the less important details.

6.3 DAF Metamodel

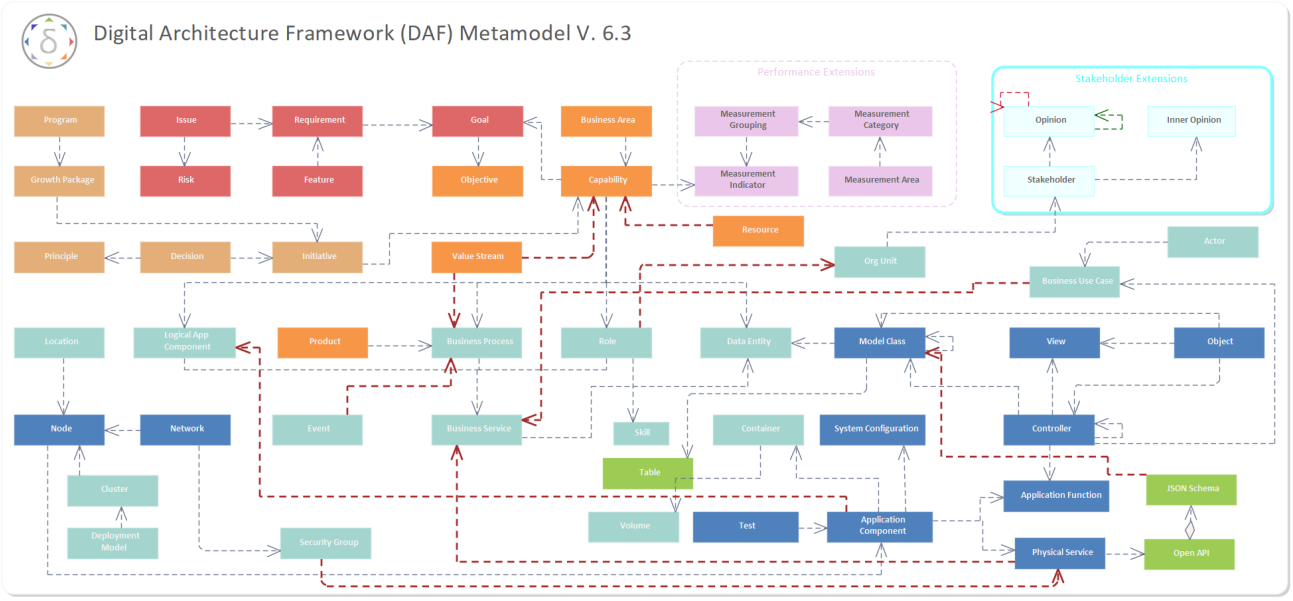


Figure 4: DAF Metamodel

The DAF Metamodel supports Enterprise Architecture, Solution Architecture, Design Architecture and more. See the description of the DAF layers below.

6.4 Example Model Instance

Each element of the Metamodel can be instantiated into a concrete model's element.

The following diagram shows an example of a DAF Metamodel instance

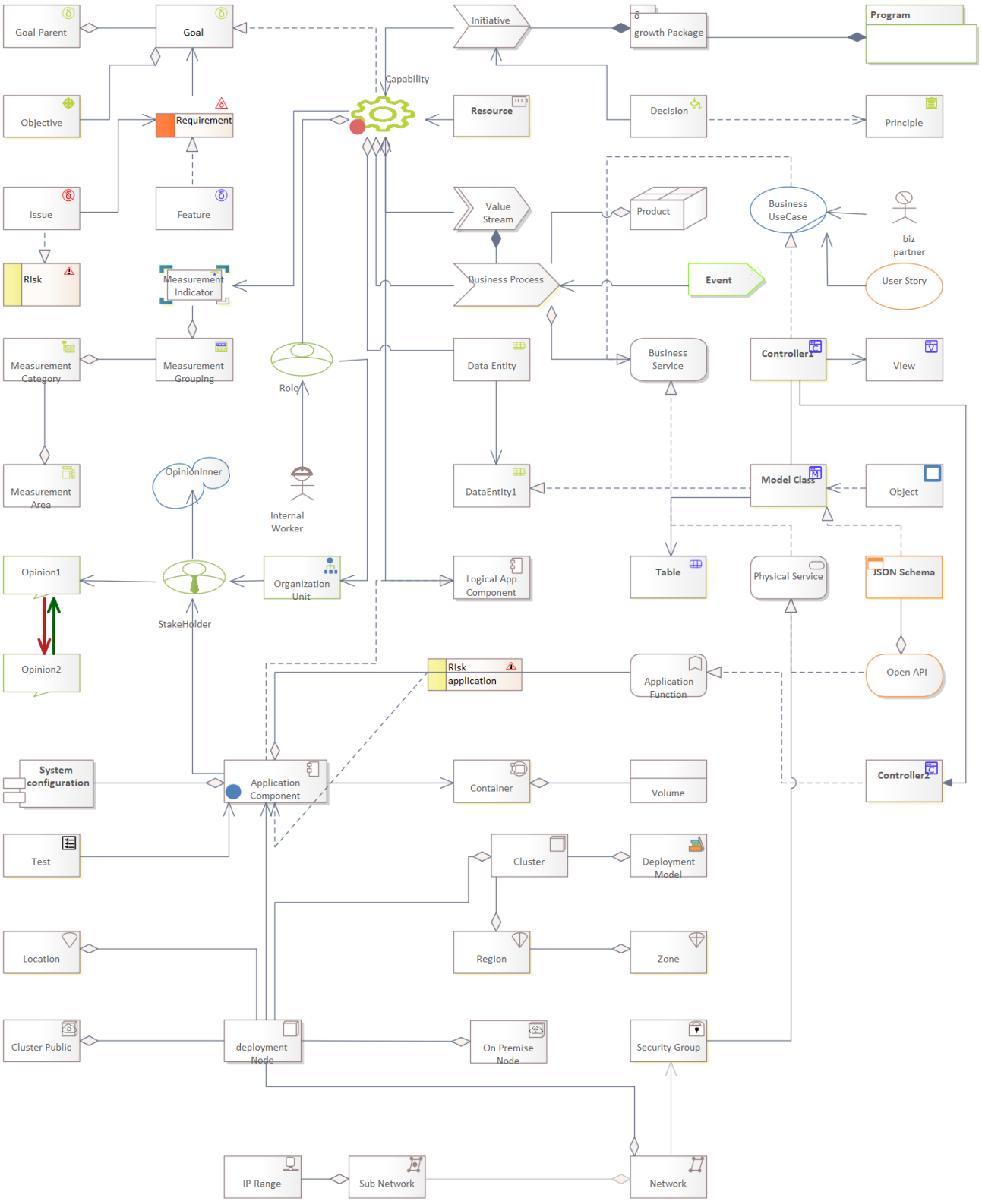


Figure 6: DAF Example Instance

DAF supports the Sparx EA quick linker that helps modelers to create only valid connections. Models can be also validated using the Checker (see below)



6.5 Layers

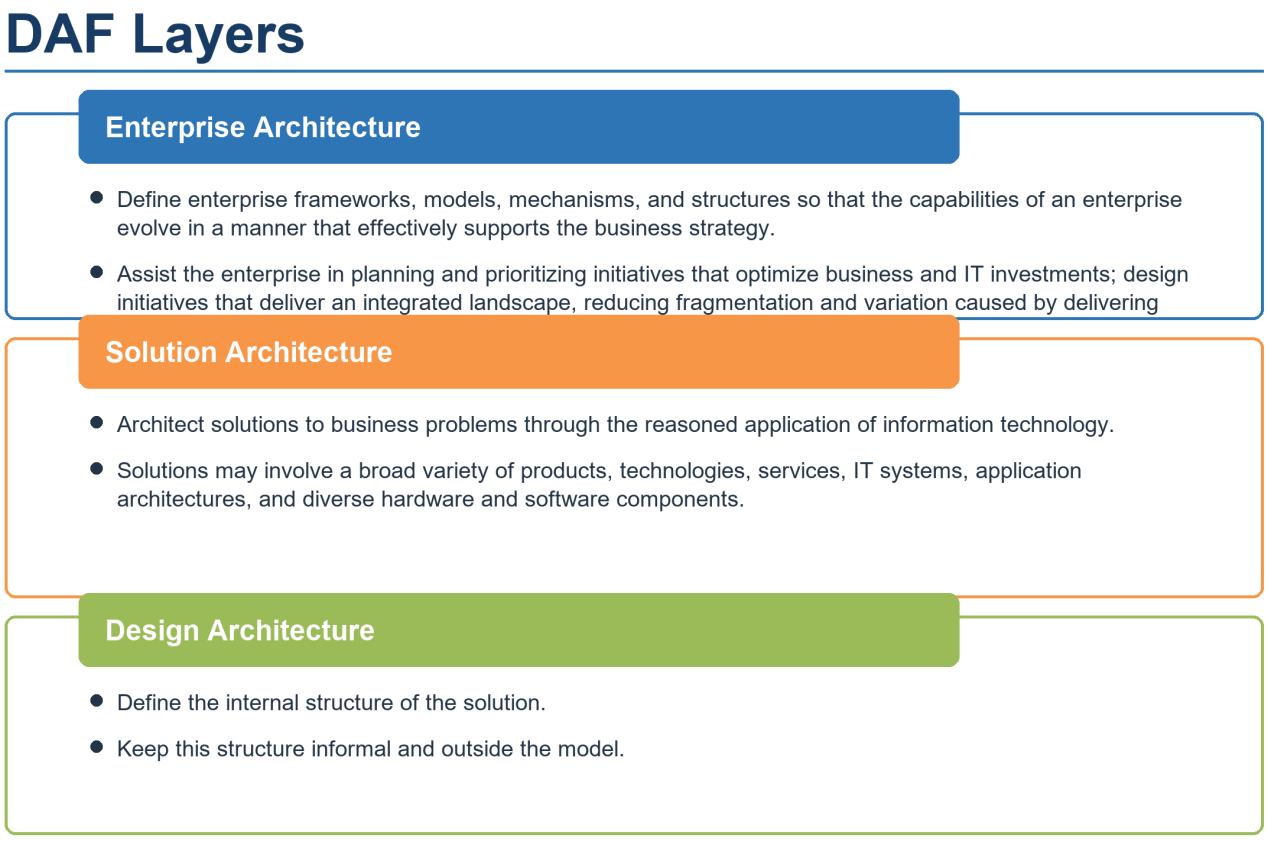


Figure 7: DAF Layers

DAF is divided into three levels of abstraction: Enterprise, Solution, and Design Architecture. Each of these levels is further broken into specific domains (see below).

6.5.1 Enterprise Architecture

- This Layer covers logical considerations. Viewing the system in terms of roles participating in processes by using resources: Who, How, and What. It corresponds to the Computational Independent Model (CIM).
- It defines enterprise frameworks, models, mechanisms, and structures so that capabilities of an enterprise evolve in a manner that effectively supports the business strategy
- Assisting the enterprise to plan and prioritize initiatives that optimize business & IT investments; designing initiatives that deliver an integrated landscape to reduces the fragmentation and variation that is the natural result of delivering solutions in an autonomous manner.
- The Enterprise Architecture level focuses on the logical considerations of the system, such as the roles, processes, and resources involved. This abstraction level helps the enterprise plan and prioritize initiatives that optimize business and IT investments, and design initiatives that deliver an integrated landscape to reduce fragmentation and variation.

6.5.2 Solution Architecture

- The Solution Architecture layer connects to the previous and translates the logical model into an operational one by describing the elements of the system that supports the business and assigns processes to them.
- It aims as definition of solutions to business problems through the reasoned application of Information Technology.

- They may involve the application and integration of a broad variety of products, technologies and services, various IT systems and applications architectures, and diverse hardware and software components.

6.5.3 Design Architecture

- This Layer designs the components of the solution, taking in account the different supporting technologies.
- The definition of the internal structure of the solution.
- Generation of Code

6.6 Domains

The abstraction layers are further organized into domains. A domain represents a **vertical organization** of concepts, incorporating related concepts at different levels of abstraction. These domains serve to organize and structure the various components and elements of the architecture in a logical and meaningful way.

The foundational domains in DAF are:

- Business
- Data
- Application
- Technology

Business Architecture	Data Architecture	Application Architecture	Technology Architecture
A representation of holistic, multi-dimensional business views of: capabilities, end-to-end value delivery, information, and organizational structure; and the relationships among these business views and strategies, products, policies, initiatives, and stakeholders. *TOGAF*	*A description of the structure and interaction of the enterprise's major types and sources of data, logical data assets, physical data assets, and data management resources.* *TOGAF*	*A description of the structure and interaction of the applications as groups of capabilities that provide key business functions and manage the data assets.* *TOGAF*	1. *A technology building block. A generic infrastructure technology that supports and enables application or data components (directly or indirectly) by providing technology services.* 2. *An encapsulation of technology infrastructure that represents a class of technology product or specific technology product.*

Figure 5: Foundational DAF domains, inspired by TOGAF

6.6.1 Business

This domain covers the business-specific concepts, such as business processes, services, and functions that an organization offers to its customers. It focuses on understanding the business objectives and strategies, and how the IT solutions can support those goals.

6.6.2 Data

This domain covers information-related concepts, such as data models, data architecture, data governance, data security, and data management. It’s focused on understanding the organization's data requirements, and how the IT solutions can support data management and access.

6.6.3 Application:

This domain covers software-related concepts, such as application architecture, application integration, and service-oriented architecture. This domain is focused on understanding the organization's application requirements, and how the IT solutions can support application deployment and management.

6.6.4 Technology

This domain covers technology-related concepts, such as infrastructure, network, security, and cloud. This domain is focused on understanding the organization's technology requirements, and how the IT solutions can support infrastructure and technology management.

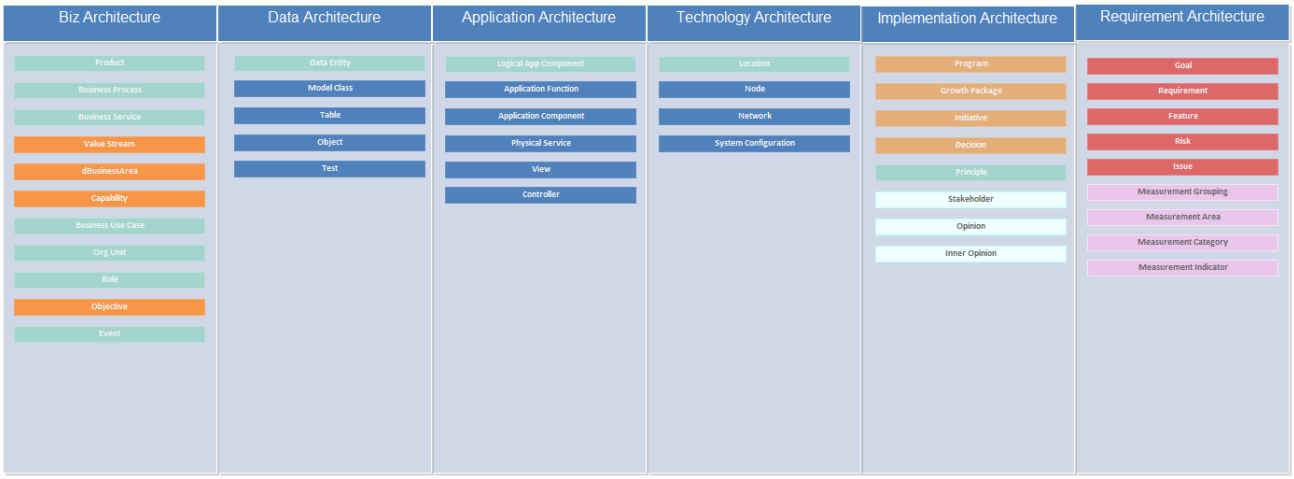


Figure 6: Domains and concepts of DAF

6.6.5 Extension Domains

Additional Domains can be also used with DAF when required.

Those include Strategy, Implementation and Requirements.

6.7 Viewpoints and Views

The intersection of a Layer and Domain creates a Viewpoint, addressing specific concerns about a system of interest. A Viewpoint is a way of looking at a system from a specific perspective and addressing specific concerns related to that system. In the context of DAF, a **Viewpoint** is the intersection of a Layer and a Domain.

Each Layer, such as Enterprise, Solution, and Design Architecture, provides a different level of abstraction and focus, while each Domain, such as Business, Data, Application, and Technology, represents a specific area of concern. By intersecting these two concepts, a Viewpoint is created that addresses specific concerns of the system-of-interest from the perspective of that Layer and Domain. For example, a Business Viewpoint within the Enterprise Layer would focus on the logical considerations of how the system supports the business strategy, while a Data Viewpoint within the Design Layer would focus on the design of the data components of the solution.

The use of Viewpoints allows for a more targeted and granular approach to analyzing and designing a system, as different stakeholders can look at the same system from different perspectives.

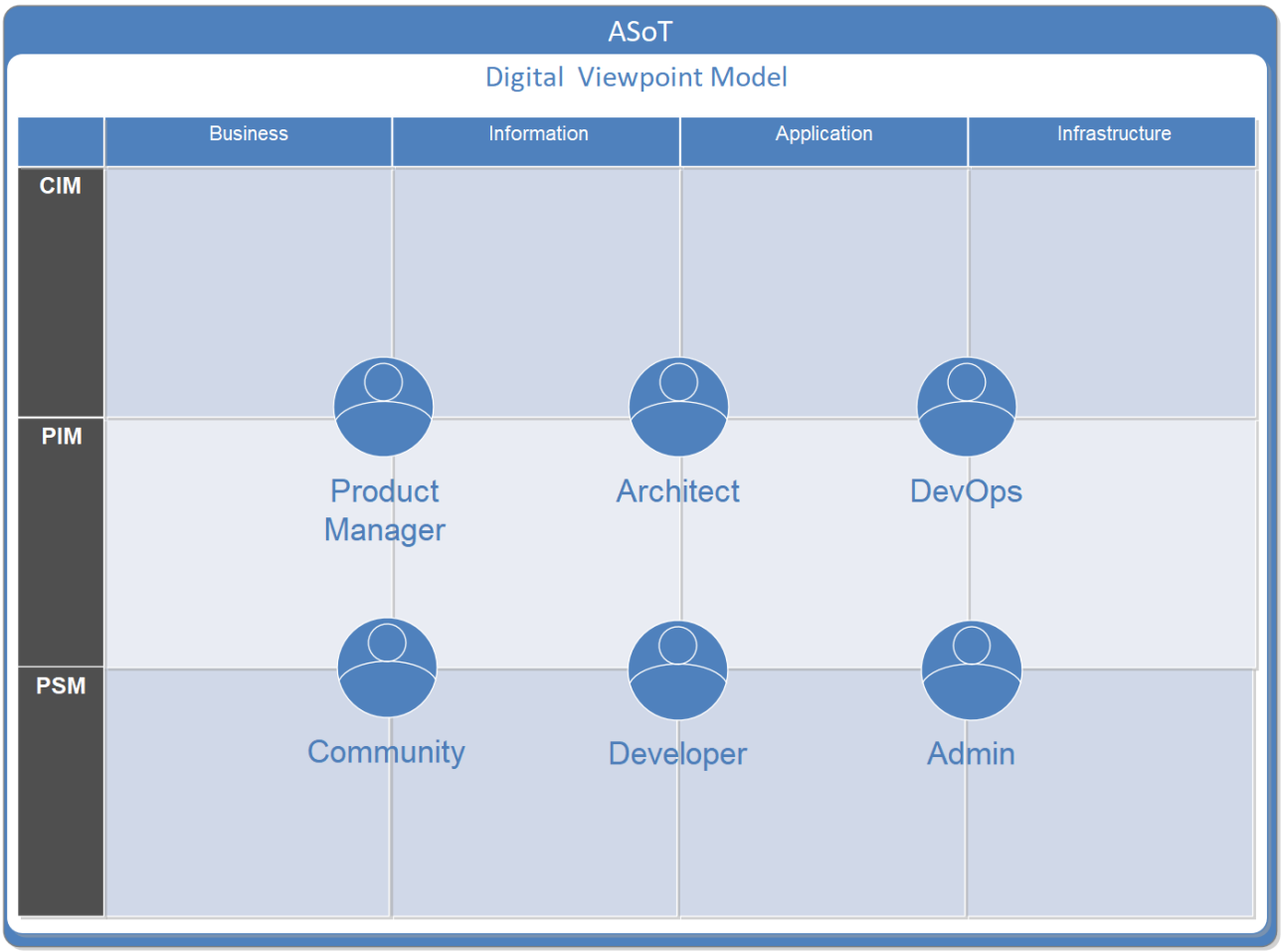


Figure 7: Digital Viewpoint Model

DAF features 12 basic viewpoints

Figure 8: example of viewpoint and their stakeholders

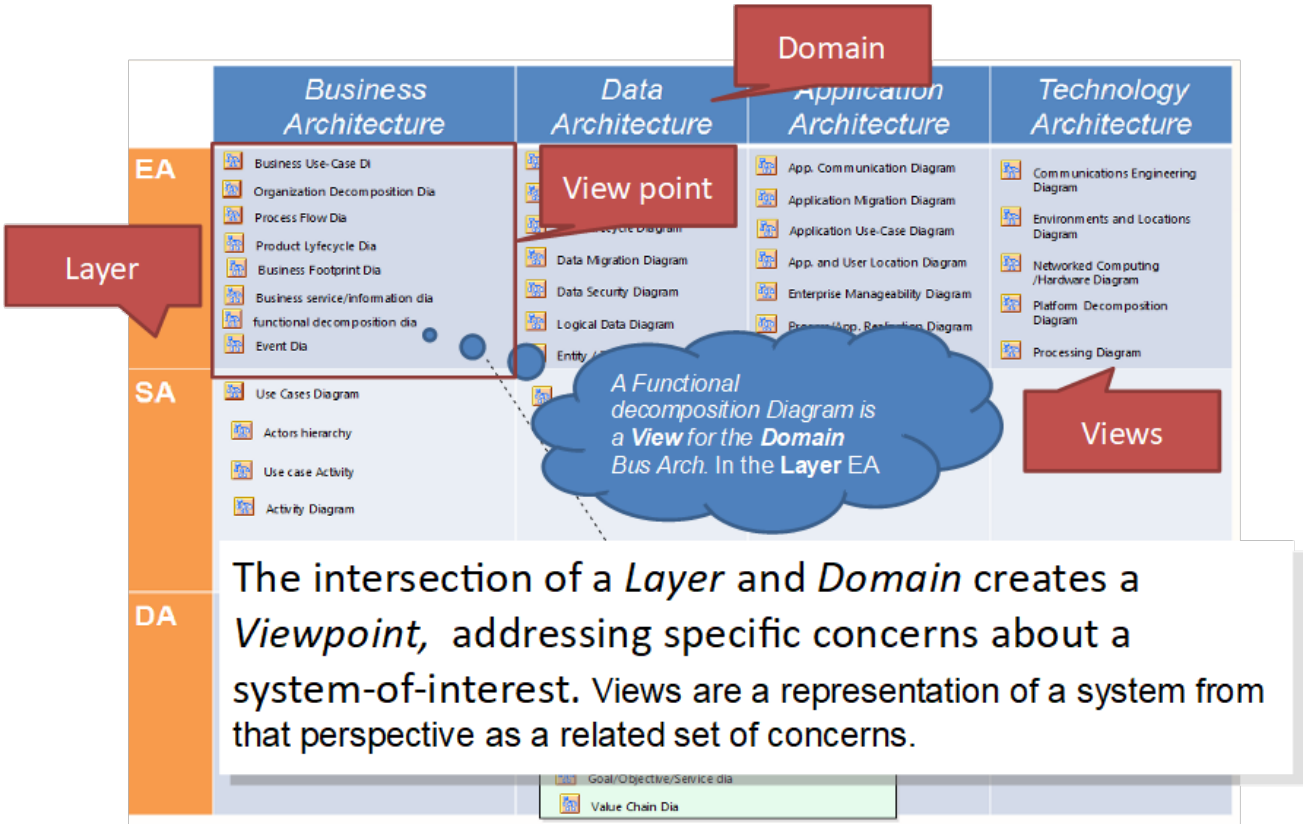


Figure 8: DAF Viewpoints

Views are representations of a system from that perspective as a related set of concerns.

6.7.1 DAF Concepts Organized in Viewpoints

Each of the concepts of the Metamodel belong to a specific Viewpoint. As such, it's easier to assign responsibility to certain roles.



Figure 9: DAF Concepts Organized in the Viewpoints

6.7.2 Views and work products

DAF supports the SPEM concept of Deliverable.

A **Deliverable** is created when a **role** performs an activity within a process that produces one or more views. Each view is built from the DAF entities stored in the repository catalog. Multiple views together form a complete deliverable.

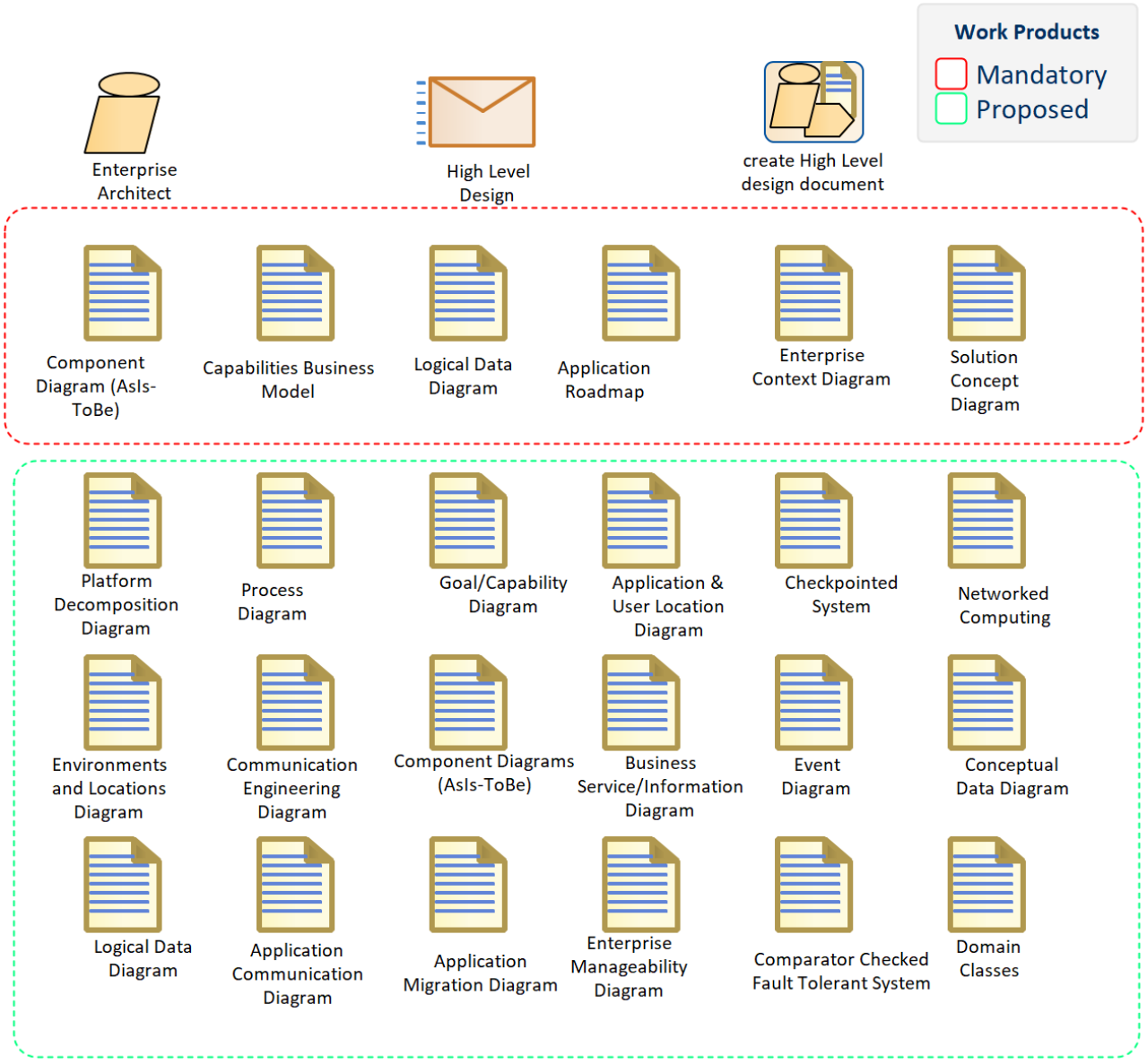


Figure 10: SPEM View with the Work Products That Compose the High-Level Design Deliverable

In the example above, the Enterprise Architect performs the activity *Create High-Level Design Document*. The result is the **High-Level Design** deliverable. This includes mandatory work products, such as the *Component Diagram* (As-Is/To-Be) and the *Capability Business Model*. Depending on the scope, it may also include optional work products, like the *Platform Decomposition Diagram* or the *Process Diagram*.

By its nature, a deliverable is like a 2D photo: it presents a static view that does not necessarily expose all implications or dependencies. However, the underlying model contains the full traceability, which can be retrieved to perform impact analysis and uncover relationships not immediately visible in the deliverable itself.

6.8 Relationships

Name	Meaning	Notation
Association	*"has"*	
Aggregation	*"shared with"*	
Composition	*"owned-by"*	
Dependency	*"uses"*	
Generalization	*"is-kind-of"*	
Realization	*"implements"*	

Figure 11: Basic UML relationships

DAF relationships are extending fundamental UML relationships:

- Association: A relationship that models a bi-directional semantic connection among instances
- Aggregation: A special kind of association that models a "whole-part" as shared relationship
- Composition: non-shareable, non-transferable ownership
- Dependency: A relationship that signifies that a single or a set of model elements requires other model elements
- Generalization: A taxonomic relationship between a more general element and a more specific element
- Realization: A specialized abstraction relationship between two sets of model elements, one representing a specification (the supplier) and the other represents an implementation of the latter (the client)

In the DAF metamodel, inheritance uses Generalizations, transitions of abstraction layers are represented with stereotyped Realization, strong connections between elements of the same abstraction level with stereotyped **Associations**, weak relationships with Dependencies; strong hierarchical relationships with **Compositions**, weak ones with **Aggregations.**

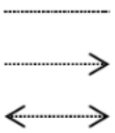
Name	Meaning	Notation
Triggering	Describes a temporal or causal relationship between elements.	
Flow	Transfer from one element to another.	
Assignment	Expresses the allocation of responsibility, performance of behavior, or execution.	
Serving	Models that an element provides its functionality to another element.	
Access	Models the ability of behavior and active structure elements to observe or act upon passive structure elements.	
Influence	Models that an element affects the implementation or achievement of some motion element.	

Figure 12: Additional ArchiMate relationships

6.8.1 Relationship Strategy

The DAF Metamodel is designed to support the concept of **encapsulation**.

Encapsulation is a key concept in modeling that refers to the practice of enclosing concept and behaviors within a single entity, so as to protect its internal state from external interference. This helps to maintain the integrity of the model by controlling the flow of information and avoiding circular references. In object-oriented modeling, encapsulation allows objects to interact with one another without being dependent on their implementation, creating a level of abstraction that makes the system more flexible and maintainable.

Circular references in modeling refer to the situation where two or more models or elements within a model refer to each other in a circular manner. This creates a situation where one element depends on another, which in turn depends on the first, leading to a never-ending chain of inter-dependencies. Circular references are often considered problematic in modeling because they can lead to ambiguities and inconsistencies in the models, making it difficult to analyze and understand the relationships between elements. They can also cause problems with automated analysis and interpretation tools that rely on the models.

Instead of circular references, **derived relationships** are used.

A derived relationship refers to a relationship that is obtained or computed from other relationships or information in the model. It is not directly specified in the model but is instead derived from existing elements and their properties and relationships. Derived relationships are often used to simplify a model and make it more manageable by abstracting complex relationships and showing only the essential relationships between elements. These relationships may also be used to generate additional information about the system being modeled or to support certain modeling activities such as impact analysis, traceability, and consistency checks.

6.9 Logic in DAF

Business logic is often treated as one thing. DAF separates three kinds of logic by where they run, and models each with its own concepts.

Kind of logic	What it is and how DAF models it
Process logic	The flow of a business process, which usually crosses several applications. Modeled with dBusinessProcess and dEvent.
Application logic	The coordination of modules or services inside one application. Modeled with dController, which implements a business use case.
Business rule	A precise condition or constraint, enforced at one point in an application. Modeled with dRule, an operation of a dController.

6.9.1 Process logic

A dBusinessProcess orchestrates the business services it uses (dProcessOrchestratesService). Its flow is explicit: dProcessFlowsToProcess links one process to the next, dEventTriggersProcess starts a process and dProcessResultsInEvent records what it produces. Process logic says what happens and in which order; it does not say how an application does its part.

6.9.2 Application logic

A dController holds the application logic of one business use case (dUseCaseisImplementedByController). The use case describes its flow of events as activities (dUseCaseContainsActivities), and each step becomes an operation of the controller. The controller governs the views the user works with (dControllerGovernsView), controls the model classes that hold the data (dControllerControlsModel) and calls other controllers through action keys (dActionKey). It implements an application function (dControllerImplementsFunction).

6.9.3 Business rules

A business rule (dRule, shown as «Rule») is a stereotyped operation, with visibility package, of the dController that enforces it. The controller is the rule set: it groups the rules that apply to its use case. The rule's Statement is plain text that a person can check and that can be passed to a large language model (LLM), which generates the implementation in a target language such as Python or TypeScript. Asking for one rule at a time, with typed inputs and a typed result, keeps the scope narrow, which reduces hallucinations and keeps rules consistent.

RuleKind records which of the four rule categories of the Object Management Group (OMG) a rule belongs to:

RuleKind	Meaning and example
Production	IF a condition on business variables holds, THEN one or more actions run. Approve Loan Application: if Applicant.CreditScore > 700 then LoanApplication.Status = Approved. Production rules (OMG PRR) do not support complex AND/OR expressions natively.
Integrity	A constraint that keeps data or a process valid and prevents invalid states. Validate Customer Age: Customer.Age >= 18 before Account.Creation; the result is a Boolean.
Derivation	Computes new information from existing facts, so derived data always matches its inputs. Calculate Discounted Price: Order.FinalPrice = Order.UnitPrice ×

RuleKind	Meaning and example
	Order.Quantity × (1 − Order.DiscountRate).
Reaction	Triggered by an event; it connects business behavior to event-driven architecture. Trigger Reorder Notification: if Inventory.Stock < Inventory.ReorderThreshold, send a reorder request to the supplier.

A rule also carries three tagged values:

- Statement: the rule in plain language, precise enough to implement, e.g. "Ensure Customer.Age >= 18 before Account.Creation".
- SourceEvidence: where the rule was found in existing code: program, paragraph or section, and line range.
- Certainty: Explicit when the rule was modeled directly, Inferred when it was extracted from existing code.

A rule's inputs and result are the operation's own parameters and return type, typed with a primitive or a class of the domain model. They are not repeated as tagged values.

A requirement of type Business Rule (dRequirement) states what must be true. A dRule says where the application enforces it, and how.

6.9.4 Modeling controllers and rules

1. In the package for the company, project, business process or use case, create a diagram, e.g. com.eurocom.crm.
2. Create a business use case and write its flow of events. In the notes of each step, state its intention in plain language.
3. Right-click the use case and select Generate Controller. The new controller refines the use case and has one operation per step.
4. Place the use case and the controller on the diagram, move the controller into the right package and set its visibility to Public.
5. Rename operations if needed. Before renaming by hand, consider changing the use case and generating the controller again.
6. Add each business rule to the controller: drag Rule from the Model page of the Digital Engineering toolbox onto the controller, or apply the dRule stereotype to an existing operation.
7. Set the operation's visibility to package and its parameters and return type to a primitive or a class of the domain model, then fill in RuleKind and Statement.

6.9.5 Rules found in existing code

When an application is modernized, rules are first extracted from the existing code, for example from IF, EVALUATE and COMPUTE statements in COBOL. Each candidate becomes a dRule with Certainty Inferred and a SourceEvidence such as "LGICUS01 / 2000-VALIDATE-AGE / L412-L431", so a reviewer can check it against the code, then confirm, merge or reject it. Certainty records where a rule came from, not whether it was reviewed.

A confirmed rule keeps its identity when it moves from the controller of the old application to the controller of the new one, so the new implementation can be traced back to the old code.

7 Detailed Documentation

The following is the complete documentation of the DAF metamodel

7.1 DAF Concepts Overview

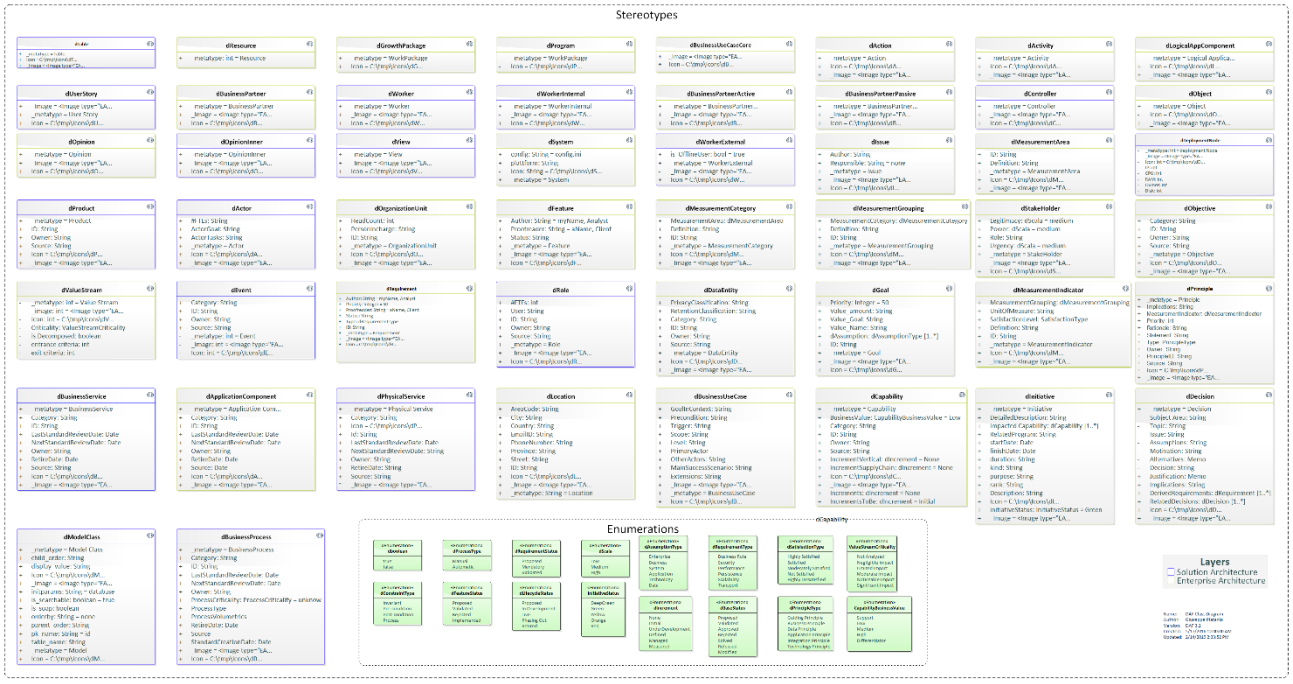


Figure 13: DAF Concepts and Properties

DAF version 7.0.0 Beta 6 supports 78 concepts (implemented as UML Stereotypes), each of those has several properties (expressed as UML Tagged Values). The table is generated from the DAF semantic model (technology/rdf/DAF-semantic-model.ttl).

Domain	Technical Name	Human Name	Documentation
			an organization of Business interests, can be decomposed.
Strategy	dBusinessArea	Business Area	
Strategy	dCapability	Capability	Represents an organizational capability to achieve business outcomes.Can be current available in an Organization Unit (or) A business-focused outcome that is delivered by the completion of one or more work packages (by increasing the capability Maturity).
			A place where business activity takes place and can be hierarchically decomposed.
Strategy	dLocation	Location	
Strategy	dMeasurementArea	Measurement Area	Represents areas of organizational or business performance measurement. The organization or Business units performance objective is captured as measurement area.
			Categorizes measurements within a measurement area.
Strategy	dMeasurementCategory	Measurement Category	

Domain	Technical Name	Human Name	Documentation
			a groups of Measurement Indicator that can be aggregated into Categories
Strategy	dMeasurementGrouping	Measurement Grouping	
			The specific measure captured for a Organization or Business Unit which can be measured and quantifiable. A measurement is a Key Performance indicator
Strategy	dMeasurementIndicator	Measurement Indicator	
			The Objective is a non material achievement, other than a Goal, the objective has no specific value to be measured. it's a time-bounded milestone for an organization used to demonstrate progress towards a goal.
Strategy	dObjective	Objective	
			A view or judgment formed about a Topic. That is formulated in an explicit fashion. Captures explicit stakeholder judgments on a topic.
Strategy	dOpinion	Opinion	
			Captures implicit stakeholder opinions related to a topic. A Stakeholder’s opinion that is relevant for the topic but is not formulated explicitly.
Strategy	dOpinionInner	Inner Opinion	

Domain	Technical Name	Human Name	Documentation
Strategy	dOrganizationUnit	Org Unit	Defines organizational units responsible for roles, can be decomposed
Strategy	dPrinciple	Principle	Defines qualitative statements guiding architecture intent.
Strategy	dProduct	Product	Encapsulates outputs produced by business processes.
Strategy	dResource	Resource	a type of Asset supporting a Business capability
Strategy	dSkill	Skill	an ability that some role possesses with various degree

Domain	Technical Name	Human Name	Documentation
Strategy	dStakeholder	Stakeholder	Represents any party or force influencing the domain of interest. Any person or force that have an interest in the considered domain. This is typically indicated by name, can be connected with a Role in the organization. A stakeholder has open and Inner opinions that are relevant for the subject matter.
Strategy	dValueStream	Value Stream	A representation of an end-to end collection of value-adding Business Processes that create an overall result for a customer, stakeholder or end user. A sequence of activities an enterprise undertakes to deliver on a customer request. More broadly, the sequence of activities required to design, produce, and deliver a good or service to a customer, and it includes the dual flows of information and material.
Business	dAction	Action	A specialized activity.
Business	dActivity	Activity	Represents business or system activities. A function or work carried out by the business user.
Business	dActor	Actor	Represents an entity (such as a person, system, or organization) that interacts with the system to achieve a specific goal. An actor typically performs actions within business Use cases and can be internal (e.g., employees) or external (e.g., customers or partner organizations), active and passive.

Domain	Technical Name	Human Name	Documentation
Business	dBusinessProcess	Business Process	A sequence of activities that produce a specific service or product supporting a particular business capability
Business	dBusinessService	Business Service	A repeatable activity; a discrete behavior that a building block may be requested or otherwise triggered to perform. A service provided by Business that achieve a business outcome in response to a request.
Business	dBusinessUseCase	Business Use Case	Encapsulates business processes producing value for an enterprise.
Business	dEvent	Event	Something that happens, inside or outside the organization, that triggers behaviour or results from it.
Business	dRole	Role	The usual or expected function of an actor, or the part somebody or something plays in a particular action or event. An actor may have a number of roles.

Domain	Technical Name	Human Name	Documentation
Requirement	dFeature	Feature	A Feature is a fact that set his related Requirement to be true
Requirement	dGoal	Goal	a Goal is a measurable scope that the organization wants to achieve. Can be hierarchically decomposed.
Requirement	dIssue	Issue	Am issue is a concrete problem that is affecting a business. Also, a fact that sets a Requirement to be false. (see also Risk)
Requirement	dRequirement	Requirement	A Business guide line about the Enterprise or the project. Requirements can be decomposed. A requirement is formulated in a SMART fashion and uses Moscow verbs (Must Have, Should Have, Could Have, and Would Like to Have.). Must Have: These are the requirements that are considered essential for the success of the initiative and must be meet for the system to function. They are often referred to as "hard" or "mandatory". requirements and are typically expressed in Boolean format as "true" or "1". Should Have: These are the requirements that are considered important, but not essential. They are typically expressed in Boolean format as "maybe" or "0/1" and are often used to prioritize the development of the system. Could Have: These are the requirements that are considered desirable but not essential. They are typically expressed in Boolean format as "false" or "0". They are used to identify potential enhancements to the system that can be added at a later date. Would Like to Have: These are the requirements that are considered less important and are often used to identify potential future enhancements to the system. In addition, the requirement should be SMART, which stands for Specific, Measurable, Achievable, Relevant, and Time-bound. SMART requirements are clear, concise, and easy to understand, and they

Domain	Technical Name	Human Name	Documentation
			provide a clear way to measure the success of the project. a risk Identifies and assesses potential issue impacting other elements. It is a potential future Issue. Can be hierarchically decomposed.
Requirement	dRisk	Risk	
			Represents decisions and their impact on architecture or processes.
Implementation	dDecision	Decision	
			A set of actions identified to achieve one or more objectives for the business. A work package can be a part of an Initiative, a complete project, or a part of a Program. A Growth pack is a collection of Initiatives architected for consistency and aimed to be executed in the same period.
Implementation	dGrowthPackage	Growth Package	
			An activity carried out by a Business Unit or organization to achieve a particular goal or improve the Maturity of a certain capability.
Implementation	dInitiative	Initiative	
			A group of Growth Packages that achieve a complex outcome for the organization
Implementation	dProgram	Program	

Domain	Technical Name	Human Name	Documentation
Implementation	dUserStory	User Story	Captures user-centric requirements expressed as stories.
Application	dApplicationComponent	Application Component	An application component is a self-contained unit. As such, it is independently deployable, re-usable, and replaceable. An application component performs one or more application functions. It encapsulates its behavior and data, exposes services, and makes them available through interfaces. Cooperating application components are connected via application collaborations. An application component may be assigned to one or more application functions. An application component has one or more application interfaces, which expose its functionality. Application interfaces of other application components may serve an application component. The name of an application component should preferably be a noun.
Application	dApplicationFunction	Application Function	Encapsulates functions supported by applications.
Application	dController	Controller	Encapsulates application behavior and flow logic.
Application	dRule	Rule	A precise business condition or constraint, implemented at a specific point in an application. Modeled as an operation with package visibility on a dController, which acts as the rule set. Its Statement is the human-readable rule passed to an LLM to generate the implementation.

Domain	Technical Name	Human Name	Documentation
Application	dLogicalAppComponent	Logical App Component	Encapsulates logical software components independent of implementation or technology.
Application	dPhysicalService	Physical Service	a technical implementation of a service that runs on physical or virtual infrastructure, providing specific functionalities, usually in the form of software applications, middleware, databases, or platforms. It is concerned with the deployment, operation, and management of the technology stack, ensuring that the service is available, performant, and reliable. Examples include web servers, APIs, databases, and network services.
Application	dTest	Test	Represents a mechanism for verifying or validating the functionality, behavior, or compliance of am application component . Tests can include automated or manual procedures designed to ensure alignment with requirements, detect defects, and confirm expected outcomes.
Application	dValue	Value	An attribute of a Node value type used in DAF. These values are persistent
Application	dValueRef	Value Ref.	A Node value type used in DAF that reference another Node Value, similar to a foreign Key. These values are persistent

Domain	Technical Name	Human Name	Documentation
Application	dView	View	A View used in DAF. Controller, Views and Associations define the application flow
Data	dModelClass	Model Class	Represent a class of the domain model, that can be converted into a Persistent domain class, must inherit from Node. It is the application's dynamic data structure
Data	dObject	Object	an Object is an instance of a class in a particular moment in runtime that can have its own state and data values.
Data	dTable	Table	Represents physical tables in a database schema.
Technology	dAPI	Open API	OpenAPI Specification is an open-source format and initiative for designing and creating machine-readable interface files that are utilized in producing, describing, consuming, and visualizing RESTful APIs and web services.

Domain	Technical Name	Human Name	Documentation
Technology	dAPIOperation	API Operation	Defines operations exposed by an API for interaction with external systems. Also called endpoints.
Technology	dAPIParameter	API Parameter	Defines parameters in API operations.
Technology	dAPIResponse	API Response	Represents responses returned by API operations.
Technology	dCluster	Cluster	Represents a set of nodes working in the same Region, akin to ArchiMate’s Node concept within a server infrastructure.
Technology	dContainer	Container	Encapsulates application components and services, mirroring ArchiMate's concept of a Node, where an application runs.

Domain	Technical Name	Human Name	Documentation
Technology	dDataEntity	Data Entity	An encapsulation of data that is recognized by a business domain expert as a thing. Logical data entities can be tied to applications, repositories, and services and may be structured according to implementation considerations.
Technology	dDeploymentModel	Deployment Model	The root class for the description of a Deployments to roll out an application, service, or container to a set of resources. akin to ArchiMate's Technology Layer deployment views.
Technology	dDeploymentNode	Node	Represents a physical machine or virtual server that hosts services, applications, or containers, akin to ArchiMate’s Node concept in the Technology Layer.
Technology	dIPRange	IP Range	Defines a range of IP addresses (CIDR) used within a network. This aligns with ArchiMate's concept of Infrastructure Service, providing connectivity.
Technology	dJSON_Attribute	JSON Attribute	Defines attributes in JSON data structures.

Domain	Technical Name	Human Name	Documentation
Technology	dJSON_Datatype	JSON Data Type	Represents a data type used within JSON structures, defining the format and constraints of data elements in a schema. JSON Data Types include standard types such as string, number, object, array, boolean, and null, ensuring consistency and validation in data exchange between systems.
Technology	dJSON_Element	JSON Element	Represents elements or nodes in JSON schemas.
Technology	dJSON_Schema	JSON Schema	The Schema Object allows the definition of input and output data types. These types can be objects, but also primitives and arrays. This object is based on the JSON Schema Specification Draft 4 and uses a predefined subset of it. On top of this subset, there are extensions provided by this specification to allow for more complete documentation
Technology	dJSON_SchemaSubSet	JSON Schema Sub Set	Represents subsets of JSON schema structures.
Technology	dJSON_Type	JSON Type	Represents data type definitions for structured information.

Domain	Technical Name	Human Name	Documentation
Technology	dNetwork	Network	The main communication medium between services and deployments, mapped to ArchiMate Communication Networks, facilitating inter-service communication.
Technology	dOnPremise	On Premise Node	an organization of Business interests
Technology	dPublicCluster	Public Cluster	A public cloud-based cluster, following the ArchiMate concept of Infrastructure Service provided by external cloud providers (Google, AWS, etc.).
Technology	dRegion	Region	A geographic area containing clusters and zones, aligned with ArchiMate's Location concept for representing deployment geography.
Technology	dSecurityGroup	Security Group	Controls and manages access and permissions between services, in line with TOGAF's Security Architecture for protecting digital assets.

Domain	Technical Name	Human Name	Documentation
Technology	dSubNetwork	Sub Network	A subset of a network, used to organize and isolate portions of the network. Equivalent to an ArchiMate Node, responsible for networking resources.
Technology	dSystem	System Configuration	A system configuration. Settings are modeled as attributes with default values, each attribute of the System Config becomes a line in the configuration file. e.g. SystemConfig .path = c:/users/app
Technology	dVolume	Volume	Refers to the storage assigned to containers, ensuring persistence, aligning with the Data Object in ArchiMate's Application Layer.
Technology	dZone	Zone	Defines macro divisions containing regions mapped to ArchiMate's Infrastructure Service.
Value types, commercial and financial	dContract	Contract	Provides an agreement between a service consumer and a service provider that establishes functional and non-functional parameters for interaction.

Domain	Technical Name	Human Name	Documentation
Value types, commercial and financial	dCostPeriod	Cost Period	Spend, or a spending cap, of one element in one period, planned or actual. Spend belongs to an initiative, application, role, capability, product or resource; a cap belongs to a program (its budget). costNature says whether it is capital or operating spend, and costFunction which line of the income statement an operating cost lands on. Spend can be included in the financial period of the organisation unit whose books it lands in (dFinancialPeriodIncludesCostPeriod).
Value types, commercial and financial	dFinancialPeriod	Financial Period	An organisation unit's reported or planned results for one period. It holds the income-statement lines (revenue, costOfRevenue, operatingExpense, depreciation, interestExpense, incomeTax), the cash line capitalExpenditure, and investment figures. Subtotals such as gross margin, EBITDA, EBIT, net income and free cash flow are derived from these lines and are never stored. The cost and revenue periods it includes show where its figures come from. Linked to its unit by dOrganizationUnitReportsFinancialPeriod. Use kind Actual on its amounts for reported results and Estimate for a plan.
Value types, commercial and financial	dQuantityValue	Quantity	Represents a measurable value expressed as a numeric amount and a unit of measurement. It provides a reusable structure for quantitative properties of DAF elements.
Value types, commercial and financial	dRevenuePeriod	Revenue Period	Revenue earned by one product or contract in one period, reported (kind Actual) or planned (kind Estimate). It can be included in the financial period of the organisation unit whose books it lands in (dFinancialPeriodIncludesRevenuePeriod).

7.2 DAF Relationship Types

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
ContractRelatedProduct	Contract is Related to Product	dContract [0..*] → dProduct [0..*]	relates to Product / is related to Contract
dAccord	According Opinion	dOpinion [1] → dOpinion [0..*]	is in accord with other Opinion / is in accord with original Opinion
dActionKey	Action Key	dController [0..*] → dController [0..*]	target Controller / Source Controller
dActivityContainsAction	Activity contains Action	dActivity [0..*] → dAction [0..*]	is contained in Activity / contains Action
dActorParticipatesInBizUseCase	Actor Participates In Use Case	dActor [0..*] → dBusinessUseCase [0..*]	Partecipates in Use Case / is executed by Actor
dAPIAggregatesSchema	Aggregates Schema	dAPI [0..*] → dJSON_Schema [0..*]	Aggregates Schema / is aggregates in API
dApplicationAggregatesFunction	aggregates Function	dApplicationComponent [0..1] → dApplicationFunction [0..*]	aggregates Function / part of Application
dApplicationComponentHasContainer	Application Has Container	dApplicationComponent [0..*] → dContainer [0..*]	Has Container / contains Application
dApplicationComponentHasCostPeriod	Application Has Cost Period	dApplicationComponent [0..1] → dCostPeriod [0..*]	has Cost Period / is cost of Application

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
dApplicationComponentRealizesLogicalComponent	Realizes Component	dApplicationComponent [0..*] → dLogicalAppComponent [0..*]	Realizes Logical component / is Realized by Component
dApplicationconfiguredbySystemConfiguration	Application configured by System Configuration	dApplicationComponent [0..*] → dSystem [0..1]	has configuration / configures Application
dApplicationConnectedWithApplication	Connects Application	dApplicationComponent [0..*] → dApplicationComponent [0..*]	connected From Application / Connected to Application
dApplicationExposesService	Application Exposes Ph. Service	dApplicationComponent [0..*] → dPhysicalService [0..*]	is exposed by Application / Exposes Phy Service
dApplicationHasOwner	Application has Owner	dApplicationComponent [0..*] → dStakeholder [0..*]	owns Application / has Owner
dApplicationRisk	Application Risk	dRisk [0..*] → dApplicationComponent [0..*]	is related to Application / has Risk
dApplicationRunsInNode	Application is run By Node	dDeploymentNode [0..*] → dApplicationComponent [0..*]	Runs Application / is run By Node
dAreaAggregatesCategories	Area aggregates Categories	dMeasurementArea [1] → dMeasurementCategory [0..*]	aggregates categories / has Measurement Area
dBizServIsDescribedByUseCase	Biz Serv Is Described By Use Case	dBusinessUseCase [0..*] → dBusinessService [0..*]	Has service / described by Use Case
dBizServUsesDataEntity	Service Uses Data	dBusinessService [0..*] → dDataEntity [0..*]	uses Data / is used by service
dBusinessAreaHasCapability	has Capability	dBusinessArea [1] → dCapability [1..*]	has Capability / is organized in Biz

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
			Area
dCapabilityHasCostPeriod	Capability Has Cost Period	dCapability [0..1] → dCostPeriod [0..*]	has Cost Period / is direct cost of Capability
dCapabilityisMeasuredByKPI	is Measured By KPI	dCapability [1..*] → dMeasurementIndicator [0..*]	is Measured By Indicator / is measuring Capability
dCategoryaggregatesGroups	aggregates Groups	dMeasurementCategory [1] → dMeasurementGrouping [0..*]	aggregates Groups / has Category
dClusterContainsDeploymentNode	Cluster Contains Node	dCluster [1] → dDeploymentNode [0..*]	Contains DeploymentNode / is Contained in Cluster
dContainerAggregatesVolume	Container Aggregates Volume	dContainer [0..*] → dVolume [0..*]	Aggregates Volume / is Aggregated in Container
dContractHasRevenuePeriod	Contract Has Revenue Period	dContract [0..1] → dRevenuePeriod [0..*]	has Revenue Period / is revenue of Contract
dContractServesActor	Contract Serves Actor	dContract [0..*] → dActor [0..*]	is serving actor / served by contract
dControllerControlsModel	Controls Model	dController [0..1] → dModelClass [0..*]	Controls Model / is controlled by Controller
dControllerGovernsView	Governs View	dController [0..1] → dView [0..*]	Governs view / is Governed by Controller
dControllerImplementsFunction	controller Implements function	dController [0..*] → dApplicationFunction [0..1]	Implements function / has Controller

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
dDataEntityInformsCapability	informs Capability	dCapability [0..*] → dDataEntity [0..*]	is informed by Data Entity / Informs Capability
dDecisionRefersToPrinciple	Decision refers To Principle	dDecision [0..*] → dPrinciple [1..*]	refers To Principle / is referenced by Decision
dDeploymentModelAggregatesCluster	Deployment Model Aggregates Cluster	dDeploymentModel [1] → dCluster [1..*]	Aggregates Cluster / is Aggregated in Deployment Model
dDiscord	Discording opinion	dOpinion [1] → dOpinion [0..*]	is in discord with other Opinion / is in discord with Original Opinion
dEntityAssociatesEntity	Entity Associates Entity	dDataEntity [0..*] → dDataEntity [0..*]	Is connected to Source Entity / Is connected to Target Entity
dEventLeadsToEvent	Event Leads to Event	dEvent [0..*] → dEvent [0..*]	leads to Event / follows Event
dEventTriggersProcess	Event Triggers Process	dEvent [0..*] → dBusinessProcess [0..*]	triggers Process / is triggered by Event
dFeatureIsRealizedByUseCase	Feature is realized by Use Case	dBusinessUseCase [0..*] → dFeature [0..*]	realizes Feature / is realized by Use Case
dFinancialPeriodIncludesCostPeriod	Financial Period Includes Cost Period	dFinancialPeriod [0..1] → dCostPeriod [0..*]	includes Cost Period / is included in Financial Period
dFinancialPeriodIncludesRevenuePeriod	Financial Period Includes Revenue Period	dFinancialPeriod [0..1] → dRevenuePeriod [0..*]	includes Revenue Period / is included in Financial Period

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
dGoalAggregatesGoal	Goal Aggregates Goal	dGoal [0..*] → dGoal [0..*]	parent Goal / Child Goal
dGoalhasObjective	Goal has Milestone	dGoal [0..*] → dObjective [0..*]	Has Objective / is related to Goal
dGoalisOperationalizedByCapability	goal is Operationalized By Capability	dCapability [0..*] → dGoal [1..*]	support Goal / is Operationalized By Capability
dGoalTargetsdMeasurementIndicator	Goal Targets KPI	dGoal [0..*] → dMeasurementIndicator [0..*]	Targets KPI / has target Goal
dGroupAggregatesKPI	Group aggregates KPI	dMeasurementGrouping [1..*] → dMeasurementIndicator [0..*]	aggregates KPI / part of a Grouping
dInitiativeChangesLogicalComponent	Initiative Changes Logical Component	dInitiative [0..*] → dLogicalAppComponent [0..*]	changes Logical Component / is changed by Initiative
dInitiativeChangesProcess	Initiative Changes Process	dInitiative [0..*] → dBusinessProcess [0..*]	changes Process / is changed by Initiative
dInitiativeDeploysApplication	Initiative Deploys Application	dInitiative [0..*] → dApplicationComponent [0..*]	deploys Application / is deployed by Initiative
dInitiativeHasCostPeriod	Initiative Has Cost Period	dInitiative [0..1] → dCostPeriod [0..*]	has Cost Period / is cost of Initiative
dInitiativeIncreasesMaturityOf	Initiative Increases Maturity Of Capability	dInitiative [0..*] → dCapability [0..*]	Increases Maturity Of Capability / is matured by Initiative
dInitiativeisMakingUseOfDecision	Initiative is Making Use Of Decision	dDecision [0..*] → dInitiative [0..*]	is used in Initiative / Makes Use Of

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
			Decision
dInitiativeRetiresApplication	Initiative Retires Application	dInitiative [0..*] → dApplicationComponent [0..*]	retires Application / is retired by Initiative
dIssueStopsRealizationOfRequirement	Issue Stops Realization Of Req	dIssue [0..*] → dRequirement [0..*]	stop Requirement / is stopped by Issue
dJSON_SchemaSubSetGeneralizes_JSON_Schema	Schema Generalizes Schema	dJSON_Schema [0..*] → dJSON_SchemaSubSet [0..*]	Parent Schema / Child Schema
dJSONElement_associates_JSONElement	dJSONElement_associates_JSONElement	dJSON_Element [0..*] → dJSON_Element [0..*]	target JsonElement / Source JsonElement
dJSONSchemaRealizesModel	Schema Realizes Model	dJSON_Schema [0..*] → dModelClass [1]	realizes Model Class / is realized by Schema
dLocationHostsNode	Location Hosts Node	dLocation [1] → dDeploymentNode [0..*]	Hosts Node / Is hosted in Location
dLogAppCompExposesBizService	exposes Biz Service	dLogicalAppComponent [0..*] → dBusinessService [0..*]	application runs service / service run by application
dLogicalApplicationComponentSupportsCapability	App Supports Capability	dCapability [1..*] → dLogicalAppComponent [0..*]	supported by Application / supports Capability
dMeasurementIndicatorImprovesMeasurementIndicator	KPI Improves KPI	dMeasurementIndicator [0..*] → dMeasurementIndicator [0..*]	Improves KPI / Improved KPI
dMeasurementIndicatorWorsenMeasurementIndicator	Worsen KPI	dMeasurementIndicator [0..*] → dMeasurementIndicator [0..*]	Worsen KPI / is worsen by

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
dModelClassAssociatesModelClass	Model Associates Model	dModelClass [0..*] → dModelClass [0..*]	Associates Target Model / Associates Source Model
dModelClassRealizesEntity	Model Class Realizes Entity	dModelClass [1] → dDataEntity [0..1]	Realizes Entity / is realized by Class
dNetworkAggregatesSubNetwork	Network Aggregates SubNetwork	dNetwork [0..*] → dSubNetwork [0..*]	Aggregates Sub-Network / is Aggregated in Network
dNetworkContainsNode	Contains Node	dNetwork [0..1] → dDeploymentNode [0..*]	Contains Node / is contained in a Network
dNetworkHasSecurityGroup	Network Has Security Group	dNetwork [0..*] → dSecurityGroup [0..*]	Has Security Group / is associated to Network
dObjectInstanceofController	Instance of Controller	dObject [0..*] → dController [0..*]	Has an Instance / Istance of Controller
dObjectInstanceofModel	Instance of Model	dObject [0..*] → dModelClass [0..*]	Has an Instance / Istance of Model
dObjectInstanceofView	Instance of View	dObject [0..*] → dView [0..*]	Has an Instance / Istance of View
dOrganizationUnitReportsFinancialPeriod	Org Unit Reports Financial Period	dOrganizationUnit [1] → dFinancialPeriod [0..*]	reports Financial Period / is reported by Org Unit
dOrgUnitHasStakeholders	Org Unit has Stakeholders	dOrganizationUnit [0..*] → dStakeholder [0..*]	has Stakeholder / is concerned with Organization
dPackageIsDeliveredByInitiative	Package is delivered by Initiative	dGrowthPackage [1] → dInitiative	is delivered by Initiative / is part of a

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
		[0..*]	Package
dPhysicalServiceImplementsBusinessService	Physical Serv. Implements Biz Serv.	dPhysicalService [0..*] → dBusinessService [0..*]	Implement Biz Service / is Implemented by physical service
dPolymorphicCollection	Polymorphic Collection	dJSON_Schema [0..*] → dJSON_Schema [0..*]	@baseType / @Type
dProcessControlsProcess	Process Controls Process	dBusinessProcess [0..*] → dBusinessProcess [0..*]	controls Process / is controlled by Process
dProcessEnsureCorrectOperationOfCapability	Ensure the correct Operation of	dCapability [0..*] → dBusinessProcess [0..*]	is operated by Process / Ensure the correct Operation of Capability
dProcessFlowsToProcess	Process Flows to Process	dBusinessProcess [0..*] → dBusinessProcess [0..*]	Sequence Flow To / Sequence Flow From
dProcessOrchestratesService	Process Orchestrates Service	dBusinessProcess [0..*] → dBusinessService [0..*]	Has Service / part of Process
dProcessResultsInEvent	Process Results in Event	dBusinessProcess [0..*] → dEvent [0..*]	results in Event / is result of Process
dProductHasCostPeriod	Product Has Cost Period	dProduct [0..1] → dCostPeriod [0..*]	has Cost Period / is cost of Product
dProductHasRevenuePeriod	Product Has Revenue Period	dProduct [0..1] → dRevenuePeriod [0..*]	has Revenue Period / is revenue of Product
dProductIncludesProcess	Product Includes Process	dProduct [0..*] → dBusinessProcess [0..*]	Includes Process / Is part of Product

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
dProgramHasBudgetPeriod	Program Has Budget Period	dProgram [0..1] → dCostPeriod [0..*]	has Budget Period / is budget of Program
dProgramIsOrganizedInPackages	Program is organized In Packages	dProgram [1] → dGrowthPackage [0..*]	is organized In Packages / part of a Program
dRegionAggregatesCluster	Region Aggregates Cluster	dRegion [1] → dCluster [0..*]	Aggregates Cluster / is Aggregated in Region
dRelatedService	Child Process	dBusinessService [0..*] → dBusinessService [0..*]	Target Service / Source Service
dRequirementContainsRequirement	Requirement Contains Req	dRequirement [0..1] → dRequirement [0..*]	is contained by Requirement / Contains Requirement
dRequirementIsRelatedToGoal	req related to Goal	dRequirement [0..*] → dGoal [0..*]	specifies Goal / is specified by Requirement
dRequirementIsSatisfiedByFeature	is Satisfied By	dFeature [0..*] → dRequirement [0..*]	realizes Requirement / is realized by Feature
dResourceHasCostPeriod	Resource Has Cost Period	dResource [0..1] → dCostPeriod [0..*]	has Cost Period / is cost of Resource
dResourceimpactedByRisk	Resource impacted by Risk	dRisk [0..*] → dResource [0..*]	impact Resouce / is impacted by Risk
dResourceSupportsCapability	Resource supports Capability	dResource [1..*] → dCapability [1..*]	Supports Capability / is supported By Resource

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
dRiskIsRelatedToRisk	Risk is related to Risk	dRisk [0..*] → dRisk [0..*]	target Risk / Source Risk
dRiskRealizedByIssue	realized by Issue	dIssue [0..1] → dRisk [0..1]	is realizing Risk / is realized by Issue
dRoleConsumesService	Role Consumes Service	dRole [0..*] → dBusinessService [0..*]	Consumes Service / is consumed by Role
dRoleExecutesCapability	role Executes Capability	dCapability [0..*] → dRole [0..*]	is executed by Role / executes Capability
dRoleHasCostPeriod	Role Has Cost Period	dRole [0..1] → dCostPeriod [0..*]	has Cost Period / is cost of Role
dRoleHasSkill	Role has skill	dRole [0..*] → dSkill [0..*]	has SKill / known by Role
dRoleisAssociatedToOrganization	Role is Associated To Organization	dRole [0..*] → dOrganizationUnit [0..1]	works in Organization / has Role
dRoleIsPerformedByActor	Role isPerformed By Actor	dActor [0..*] → dRole [0..*]	has Role / is executed by Actor
dRoleMaintainsApplication	Role Maintains Application	dRole [0..*] → dApplicationComponent [0..*]	maintains Application / is maintained by Role
dRoleStaffsService	Role Staffs Service	dRole [0..*] → dBusinessService [0..*]	staffs Service / is staffed by Role
dSchema_Associates_Schema	Schema Association	dJSON_Schema [0..*] →	Target Schema / Source Schema

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
		dJSON_Schema [0..*]	
dSchemaAssociatesElement	Schema Associates Element	dJSON_Schema [0..*] → dJSON_Element [0..*]	Associates Element / Associates Schema
dSchemaGeneralizesSchema	Schema Generalization	dJSON_Schema [0..*] → dJSON_Schema [0..*]	Child Schema / Parent Schema
dSecurityGroupHasPhysicalService	Security Group Has Service	dSecurityGroup [0..*] → dPhysicalService [0..*]	has Physical Service / Asociated to Sec. group
dServiceRealizedByAPI	Service Realized By API	dAPI [0..*] → dPhysicalService [0..1]	realized Physical Service / is realized By API
dStakeholderHasInnerOpinion	Stakeholder Has Inner Opinion	dStakeholder [1] → dOpinionInner [0..*]	Has Inner Opinion / is thinked by Stakeholder
dStakeholderHasOpinion	Stakeholder Has Opinion	dStakeholder [1] → dOpinion [0..*]	Has Opinion / is expressed by Stakeholder
dSubNetworkAggregatesIPRange	SubNetwork Aggregates IP Range	dSubNetwork [0..*] → dIPRange [0..*]	Aggregates IP Range / is Aggregated in sub Net.
dTableProvidesPersistenceForModelClass	Table Provides Persistence to Model	dModelClass [1] → dTable [0..*]	is Saved in a Table / Provides Persistence For Model
dTest_ApplicationComponent	Test validates Component	dTest [0..*] → dApplicationComponent [0..*]	Test validates Component / is validated by test
dUseCaseContainsActivities	Use Case Contains Activities	dBusinessUseCase [0..*] → dActivity [0..*]	has Activitiy / part of UseCase

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
dUseCaseHasStory	Use Case Has Story	dUserStory [0..*] → dBusinessUseCase [0..*]	Is associated to Use Case / Can be told as User Story
dUseCaseisImplementedByController	Use Case is Implemented By Controller	dController [0..*] → dBusinessUseCase [0..*]	implements Use Case / Is implemented by Controller
dValueStreamEnablesCapability	Value Stream Enables Capability	dValueStream [0..*] → dCapability [0..*]	Enables Capability / is Enabled by Value Stream
dValueStreamIncludesProcesses	Value Stream Includes Processes	dValueStream [0..*] → dBusinessProcess [0..*]	Includes Process / is part of Value Stream
dZoneAggregatesRegion	Zone Aggregates Region	dZone [1] → dRegion [0..*]	Aggregates Region / is Aggregated in Zone

7.2.1 DAF Relationships Quick link

Source → Relationship → Target	Forward role / backward role
`dActivity → dActivityContainsAction → dAction`	`is contained in Activity` `contains Action`
`dActor → dActorParticipatesInBizUseCase → dBusinessUseCase`	`Partecipates in Use Case` `is executed by Actor`
`dActor → dRoleIsPerformedByActor → dRole`	`has Role` `is executed by Actor`
`dAPI → dAPIAggregatesSchema → dJSON_Schema`	`Aggregates Schema` `is aggregates in API`
`dAPI → dServiceRealizedByAPI → dPhysicalService`	`realized Physical Service` `is realized By API`
`dApplicationComponent → dApplicationConnectedWithApplication →`	`connected From Application`

Source → Relationship → Target	Forward role / backward role
dApplicationComponent	Connected to Application
dApplicationComponent → dApplicationAggregatesFunction → dApplicationFunction	aggregates Function part of Application
dApplicationComponent → dApplicationComponentHasContainer → dContainer	Has Container contains Application
dApplicationComponent → dApplicationComponentHasCostPeriod → dCostPeriod	has Cost Period is cost of Application
dApplicationComponent → dApplicationComponentRealizesLogicalComponent → dLogicalAppComponent	Realizes Logical component is Realized by Component
dApplicationComponent → dApplicationExposesService → dPhysicalService	is exposed by Application Exposes Phy Service
dApplicationComponent → dApplicationHasOwner → dStakeholder	owns Application has Owner
dApplicationComponent → dApplicationconfiguredbySystemConfiguration → dSystem	has configuration configures Application
dBusinessArea → dBusinessAreaHasCapability → dCapability	has Capability is organized in Biz Area
dBusinessProcess → dProcessControlsProcess → dBusinessProcess	controls Process is controlled by Process
dBusinessProcess → dProcessFlowsToProcess → dBusinessProcess	Sequence Flow To Sequence Flow From
dBusinessProcess → dProcessOrchestratesService → dBusinessService	Has Service part of Process
dBusinessProcess → dProcessResultsInEvent → dEvent	results in Event is result of Process
dBusinessService → dRelatedService → dBusinessService	Target Service Source Service
dBusinessService → dBizServUsesDataEntity → dDataEntity	uses Data is used by service

Source → Relationship → Target	Forward role / backward role
dBusinessUseCase → dUseCaseContainsActivities → dActivity	has Activitiy part of UseCase
dBusinessUseCase → dBizServIsDescribedByUseCase → dBusinessService	Has service described by Use Case
dBusinessUseCase → dFeatureIsRealizedByUseCase → dFeature	realizes Feature is realized by Use Case
dCapability → dProcessEnsureCorrectOperationOfCapability → dBusinessProcess	is operated by Process Ensure the correct Operation of Capability
dCapability → dCapabilityHasCostPeriod → dCostPeriod	has Cost Period is direct cost of Capability
dCapability → dDataEntityInformsCapability → dDataEntity	is informed by Data Entity Informs Capability
dCapability → dGoalisOperationalizedByCapability → dGoal	support Goal is Operationalized By Capability
dCapability → dLogicalApplicationComponentSupportsCapability → dLogicalAppComponent	supported by Application supports Capability
dCapability → dCapabilityisMeasuredByKPI → dMeasurementIndicator	is Measured By Indicator is measuring Capability
dCapability → dRoleExecutesCapability → dRole	is executed by Role executes Capability
dCluster → dClusterContainsDeploymentNode → dDeploymentNode	Contains DeploymentNode is Contained in Cluster
dContainer → dContainerAggregatesVolume → dVolume	Aggregates Volume is Aggregated in Container
dContract → dContractServesActor → dActor	is serving actor served by contract
dContract → ContractRelatedProduct → dProduct	relates to Product is related to Contract
dContract → dContractHasRevenuePeriod → dRevenuePeriod	has Revenue Period

Source → Relationship → Target	Forward role / backward role
	is revenue of Contract
dController → dControllerImplementsFunction → dApplicationFunction	Implements function has Controller
dController → dUseCaseisImplementedByController → dBusinessUseCase	implements Use Case Is implemented by Controller
dController → dActionKey → dController	target Controller Source Controller
dController → dControllerControlsModel → dModelClass	Controls Model is controlled by Controller
dController → dControllerGovernsView → dView	Governs view is Governed by Controller
dDataEntity → dEntityAssociatesEntity → dDataEntity	Is connected to Source Entity Is connected to Target Entity
dDecision → dInitiativeisMakingUseOfDecision → dInitiative	is used in Initiative Makes Use Of Decision
dDecision → dDecisionRefersToPrinciple → dPrinciple	refers To Principle is referenced by Decision
dDeploymentModel → dDeploymentModelAggregatesCluster → dCluster	Aggregates Cluster is Aggregated in Deployment Model
dDeploymentNode → dApplicationRunsInNode → dApplicationComponent	Runs Application is run By Node
dEvent → dEventTriggersProcess → dBusinessProcess	triggers Process is triggered by Event
dEvent → dEventLeadsToEvent → dEvent	leads to Event follows Event
dFeature → dRequirementIsSatisfiedByFeature → dRequirement	realizes Requirement is realized by Feature
dFinancialPeriod → dFinancialPeriodIncludesCostPeriod → dCostPeriod	includes Cost Period is included in Financial Period

Source → Relationship → Target	Forward role / backward role
dFinancialPeriod → dFinancialPeriodIncludesRevenuePeriod → dRevenuePeriod	includes Revenue Period is included in Financial Period
dGoal → dGoalAggregatesGoal → dGoal	parent Goal Child Goal
dGoal → dGoalTargetsdMeasurementIndicator → dMeasurementIndicator	Targets KPI has target Goal
dGoal → dGoalhasObjective → dObjective	Has Objective is related to Goal
dGrowthPackage → dPackageIsDeliveredByInitiative → dInitiative	is delivered by Initiative is part of a Package
dInitiative → dInitiativeDeploysApplication → dApplicationComponent	deploys Application is deployed by Initiative
dInitiative → dInitiativeRetiresApplication → dApplicationComponent	retires Application is retired by Initiative
dInitiative → dInitiativeChangesProcess → dBusinessProcess	changes Process is changed by Initiative
dInitiative → dInitiativeIncreasesMaturityOf → dCapability	Increases Maturity Of Capability is matured by Initiative
dInitiative → dInitiativeHasCostPeriod → dCostPeriod	has Cost Period is cost of Initiative
dInitiative → dInitiativeChangesLogicalComponent → dLogicalAppComponent	changes Logical Component is changed by Initiative
dIssue → dIssueStopsRealizationOfRequirement → dRequirement	stop Requirement is stopped by Issue
dIssue → dRiskRealizedByIssue → dRisk	is realizing Risk is realized by Issue
dJSON_Element → dJSONElement_associates_JSONElement → dJSON_Element	target JsonElement Source JsonElement
dJSON_Schema → dSchemaAssociatesElement → dJSON_Element	Associates Element

Source → Relationship → Target	Forward role / backward role
	Associates Schema
dJSON_Schema → dPolymorphicCollection → dJSON_Schema	@baseType @Type
dJSON_Schema → dSchema_Associates_Schema → dJSON_Schema	Target Schema Source Schema
dJSON_Schema → dSchemaGeneralizesSchema → dJSON_Schema	Child Schema Parent Schema
dJSON_Schema → dJSON_SchemaSubSetGeneralizes_JSON_Schema → dJSON_SchemaSubSet	Parent Schema Child Schema
dJSON_Schema → dJSONSchemaRealizesModel → dModelClass	realizes Model Class is realized by Schema
dLocation → dLocationHostsNode → dDeploymentNode	Hosts Node Is hosted in Location
dLogicalAppComponent → dLogAppCompExposesBizService → dBusinessService	application runs service service run by application
dMeasurementArea → dAreaAggregatesCategories → dMeasurementCategory	aggregates categories has Measurement Area
dMeasurementCategory → dCategoryaggregatesGroups → dMeasurementGrouping	aggregates Groups has Category
dMeasurementGrouping → dGroupAggregatesKPI → dMeasurementIndicator	aggregates KPI part of a Grouping
dMeasurementIndicator → dMeasurementIndicatorImprovesMeasurementIndicator → dMeasurementIndicator	Improves KPI Improved KPI
dMeasurementIndicator → dMeasurementIndicatorWorsenMeasurementIndicator → dMeasurementIndicator	Worsen KPI is worsen by
dModelClass → dModelClassRealizesEntity → dDataEntity	Realizes Entity is realized by Class
dModelClass → dModelClassAssociatesModelClass → dModelClass	Associates Target Model Associates Source Model

Source → Relationship → Target	Forward role / backward role
dModelClass → dTableProvidesPersistenceForModelClass → dTable	is Saved in a Table Provides Persistence For Model
dNetwork → dNetworkContainsNode → dDeploymentNode	Contains Node is contained in a Network
dNetwork → dNetworkHasSecurityGroup → dSecurityGroup	Has Security Group is associated to Network
dNetwork → dNetworkAggregatesSubNetwork → dSubNetwork	Aggregates Sub-Network is Aggregated in Network
dObject → dObjectInstanceofController → dController	Has an Instance Istance of Controller
dObject → dObjectInstanceofModel → dModelClass	Has an Instance Istance of Model
dObject → dObjectInstanceofView → dView	Has an Instance Istance of View
dOpinion → dAccord → dOpinion	is in accord with other Opinion is in accord with original Opinion
dOpinion → dDiscord → dOpinion	is in discord with other Opinion is in discord with Original Opinion
dOrganizationUnit → dOrganizationUnitReportsFinancialPeriod → dFinancialPeriod	reports Financial Period is reported by Org Unit
dOrganizationUnit → dOrgUnitHasStakeholders → dStakeholder	has Stakeholder is concerned with Organization
dPhysicalService → dPhysicalServiceImplementsBusinessService → dBusinessService	Implement Biz Service is Implemented by physical service
dProduct → dProductIncludesProcess → dBusinessProcess	Includes Process Is part of Product
dProduct → dProductHasCostPeriod → dCostPeriod	has Cost Period is cost of Product
dProduct → dProductHasRevenuePeriod → dRevenuePeriod	has Revenue Period

Source → Relationship → Target	Forward role / backward role
	is revenue of Product
dProgram → dProgramHasBudgetPeriod → dCostPeriod	has Budget Period is budget of Program
dProgram → dProgramIsOrganizedInPackages → dGrowthPackage	is organized In Packages part of a Program
dRegion → dRegionAggregatesCluster → dCluster	Aggregates Cluster is Aggregated in Region
dRequirement → dRequirementIsRelatedToGoal → dGoal	specifies Goal is specified by Requirement
dRequirement → dRequirementContainsRequirement → dRequirement	is contained by Requirement Contains Requirement
dResource → dResourceSupportsCapability → dCapability	Supports Capability is supported By Resource
dResource → dResourceHasCostPeriod → dCostPeriod	has Cost Period is cost of Resource
dRisk → dApplicationRisk → dApplicationComponent	is related to Application has Risk
dRisk → dResourceimpactedByRisk → dResource	impact Resouce is impacted by Risk
dRisk → dRiskIsRelatedToRisk → dRisk	target Risk Source Risk
dRole → dRoleMaintainsApplication → dApplicationComponent	maintains Application is maintained by Role
dRole → dRoleConsumesService → dBusinessService	Consumes Service is consumed by Role
dRole → dRoleStaffsService → dBusinessService	staffs Service is staffed by Role
dRole → dRoleHasCostPeriod → dCostPeriod	has Cost Period is cost of Role

Source → Relationship → Target	Forward role / backward role
dRole → dRoleisAssociatedToOrganization → dOrganizationUnit	works in Organization has Role
dRole → dRoleHasSkill → dSkill	has SKill known by Role
dSecurityGroup → dSecurityGroupHasPhysicalService → dPhysicalService	has Physical Service Asociated to Sec. group
dStakeholder → dStakeholderHasOpinion → dOpinion	Has Opinion is expressed by Stakeholder
dStakeholder → dStakeholderHasInnerOpinion → dOpinionInner	Has Inner Opinion is thinked by Stakeholder
dSubNetwork → dSubNetworkAggregatesIPRange → dIPRange	Aggregates IP Range is Aggregated in sub Net.
dTest → dTest_ApplicationComponent → dApplicationComponent	Test validates Component is validated by test
dUserStory → dUseCaseHasStory → dBusinessUseCase	Is associated to Use Case Can be told as User Story
dValueStream → dValueStreamIncludesProcesses → dBusinessProcess	Includes Process is part of Value Stream
dValueStream → dValueStreamEnablesCapability → dCapability	Enables Capability is Enabled by Value Stream
dZone → dZoneAggregatesRegion → dRegion	Aggregates Region is Aggregated in Zone

7.3 Visual Characteristics of some DAF stereotypes

Some DAF stereotypes have special visual characteristics. They can react to Tagged Values or other properties, e.g. diagram properties as shown below.

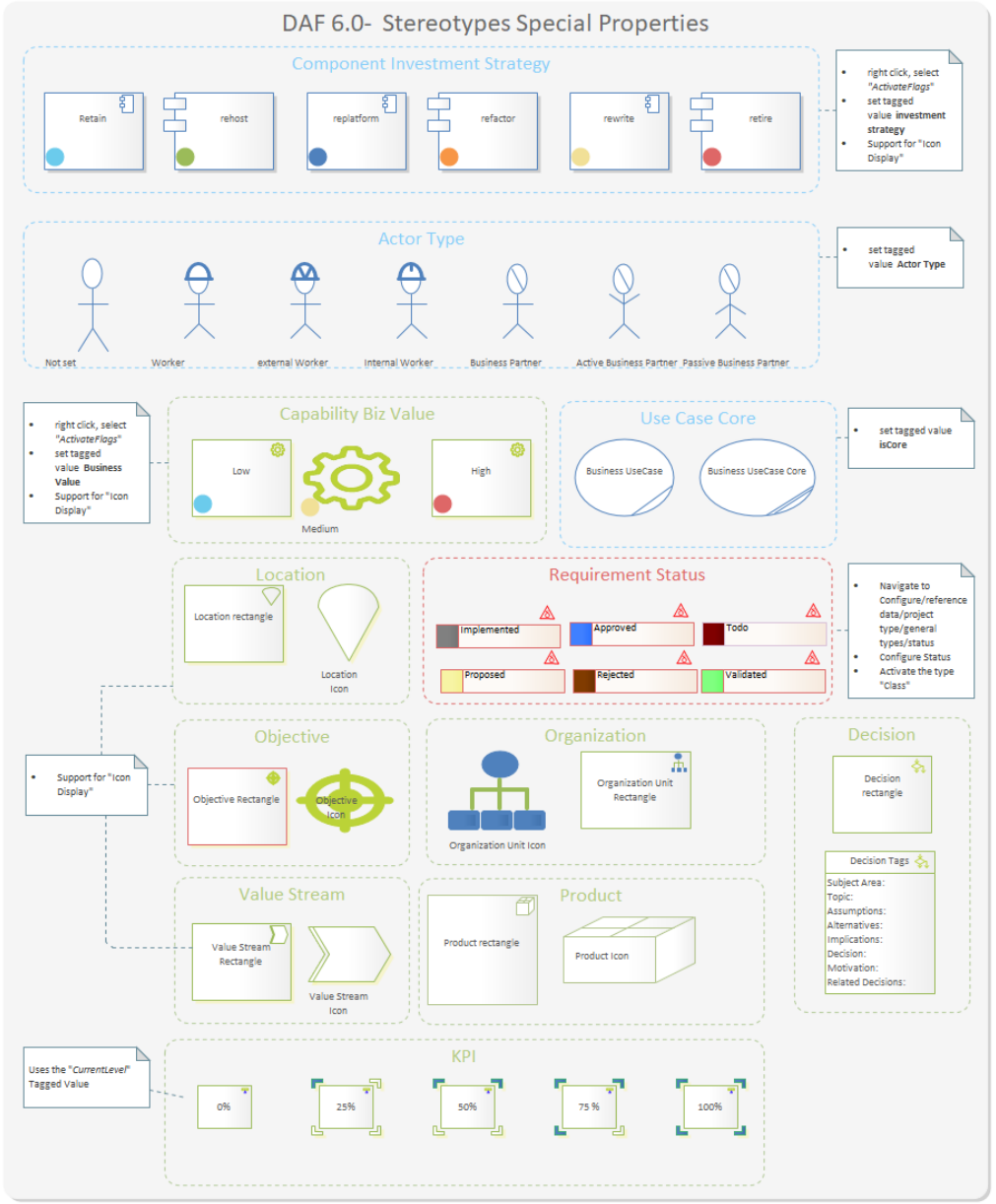


Figure 18: DAF Visual Properties

8 Appendices

8.1 Stereotypes and Tagged Values

Stereotype	Tagged Value : Type [multiplicity]	Notes
dAction		A specialized activity.
dActivity		Represents business or system activities. A function or work carried out by the business user.
dActor		Represents an entity (such as a person, system, or organization) that interacts with the system to achieve a specific goal. An actor typically performs actions within business Use cases and can be internal (e.g., employees) or external (e.g., customers or partner organizations), active and passive.
	ActorGoal : String [0..1]	What this actor wants to achieve when it deals with the organisation, in one or two sentences, e.g. "get a claim paid quickly".
	ActorTasks : String [0..1]	The main tasks this actor performs, in general terms, e.g. "files claims, uploads documents". Detailed behaviour belongs in use cases and processes.
	ActorType : ActorTypes [0..1]	The kind of actor: a worker (internal or external) or a business partner (active or passive). Absent means not known; Not Set says the same and is kept for older models.
	Count : dQuantityValue [0..1]	How many parties act as this actor. Use unit {FTE} for staff and a party unit such as {customer} or {policy} for a customer segment. Absent means not counted.
dAPI		OpenAPI Specification is an open-source format and initiative for designing and creating machine-readable interface files that are utilized in producing, describing, consuming, and visualizing RESTful APIs and web services.
	basePath : String [0..1]	The path, relative to the host, under which all operations are served. It starts with a slash, e.g. /v1. Use / when the API is served at the root of the host.
	consumes : MimeTypes [0..*]	The MIME types every operation accepts by default, e.g. application/json. An operation can override it.
	contact : Contact [0..1]	Who to contact about the API: name, e-mail and URL.
	definitions : String [0..1]	The data types the operations send and receive, as a reference to the schema that defines them, e.g. the name of a dJSON_Schema.
	Full Name : String [0..1]	The full, human-readable name of the API, e.g. "Claims Intake API".
	host : String [0..1]	The host name or IP address that serves the API, optionally with a port, e.g. api.example.com:8443. No scheme and no path. Absent means the host that serves the documentation.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	license : String [0..1]	The licence the API is published under, as a name or URL, e.g. "Apache-2.0".
	parameters : String [0..1]	Parameters shared by several operations, defined once here and referenced by the operations.
	produces : MimeTypes [0..*]	The MIME types every operation returns by default, e.g. application/json. An operation can override it.
	responses : String [0..1]	Responses shared by several operations, defined once here and referenced by the operations.
	schemes : SchemeType [0..*]	The transfer protocols the API supports (http, https, ws, wss). Absent means the scheme used to read the API description.
	securityDefinitions : String [0..1]	The security schemes the API uses, e.g. "OAuth2 authorization code", "API key in header X-Api-Key".
	swagger : String [1]	The OpenAPI (Swagger) specification version the API description follows, e.g. "2.0". Tools use it to read the rest of the description.
	tags : String [0..1]	Comma-separated tags that group the API's operations, e.g. "claims, policies".
dAPIOperation		Defines operations exposed by an API for interaction with external systems. Also called endpoints.
	base_Operation : String [0..1]	The UML operation this API operation is generated from or extends, when the API is derived from a class model.
	consumes : String [0..1]	The MIME types this operation accepts, comma-separated, e.g. application/json. Empty means the API's default.
	deprecated : Boolean [0..1]	true when the operation is deprecated and clients should stop using it; otherwise false.
	description : String [0..1]	A longer explanation of the operation's behaviour, rules and side effects.
	hTTPMethod : HTTPMethod [1]	The HTTP method of the operation, e.g. get to read, post to create, put or patch to change, delete to remove.
	produces : String [0..1]	The MIME types this operation returns, comma-separated, e.g. application/json. Empty means the API's default.
	relativePath : String [1]	The path of the operation relative to the API's basePath, with path parameters in braces, e.g. /claims/{claimId}.
	schemes : String [0..1]	The transfer protocols for this operation, e.g. https. Empty means the API's default.
	summary : String [0..1]	A one-line summary of what the operation does, e.g. "Register a new claim".

Stereotype	Tagged Value : Type [multiplicity]	Notes
	tags : String [0..1]	Comma-separated tags that group this operation with others in documentation, e.g. "claims".
dAPIParameter		Defines parameters in API operations.
	allowEmptyValue : Boolean [0..1]	true when a query or form parameter may be sent with an empty value; otherwise false.
	base_Parameter : String [0..1]	The UML parameter this API parameter is generated from or extends, when the API is derived from a class model.
	collectionFormat : String [0..1]	How an array parameter is serialised: csv, ssv, tsv, pipes or multi. Empty for parameters that are not arrays.
	location : String [1]	Where the parameter is passed: path, query, header, body or formData.
	required : Boolean [0..1]	true when the client must send the parameter; otherwise false. Path parameters are always required.
dAPIResponse		Represents responses returned by API operations.
	base_Parameter : String [0..1]	The UML element this response is generated from or extends, when the API is derived from a class model.
	default : Boolean [0..1]	true when this is the default response, used for any status code not described separately; otherwise false.
	examples : String [0..1]	Example payloads for the response, keyed by MIME type.
	headers : String [0..1]	The headers returned with the response, with their types, e.g. "Location: string".
	Integer : Int32 [1]	The HTTP status code of the response, e.g. 200, 201, 404.
dApplicationComponent		An application component is a self-contained unit. As such, it is independently deployable, re-usable, and replaceable. An application component performs one or more application functions. It encapsulates its behavior and data, exposes services, and makes them available through interfaces. Cooperating application components are connected via application collaborations. An application component may be assigned to one or more application functions. An application component has one or more application interfaces, which expose its functionality. Application interfaces of other application components may serve an application component. The name of an application component should preferably be a noun.
	ApplicationType : dApplicationType [0..1]	The kind of application, e.g. COTS for a bought package, Custom for one built in-house, DB for a database.
	BizCriticality : Int32 [0..1]	How critical the application is to the business, on a scale of 1 (can be switched off) to 100 (the business

Stereotype	Tagged Value : Type [multiplicity]	Notes
		stops without it).
	BizSatisfaction : Int32 [0..1]	How satisfied the business is with the application, on a scale of 1 (very unhappy) to 100 (fully satisfied).
	Category : String [0..1]	A free categorisation of the application, following your own taxonomy, e.g. "Core insurance", "Back office".
	depreciationMethod : dDepreciationMethod [0..1]	How its Capex is spread over usefulLife: StraightLine or DecliningBalance. The depreciation schedule is derived from the Capex cost periods, inServiceDate, usefulLife and this method; it is not stored.
	inServiceDate : Date [0..1]	The date the asset went into use; depreciation starts here. Absent means not recorded.
	InvestmentStrategy : dApplicationStrategy [0..1]	What you plan to do with the application: Retain, Rehost, Replatform, Refactor, Rewrite or Retire.
	ITSatisfaction : Int32 [0..1]	How satisfied IT is with the application, on a scale of 1 to 100. Low values usually signal technical debt.
	LastStandardReviewDate : DateTime [0..1]	When the application was last reviewed against the architecture standards.
	NextStandardReviewDate : DateTime [0..1]	When the application is next due for review against the architecture standards.
	Owner : String [0..1]	The person or team accountable for the application.
	Port : String [0..1]	The network port the application listens on, e.g. 8443, when it exposes one.
	RetireDate : Date [0..1]	The date the application is, or is planned to be, switched off. When an initiative retires the application, use that initiative's finish date or later. Absent means no retirement is planned.
	Source : String [0..1]	Where the information about this application came from, e.g. a document, interview or tool export.
	usefulLife : Duration [0..1]	How long the asset is depreciated over, as an xsd:duration, e.g. P5Y. Absent means the asset is not capitalised or the life is not recorded.
	Invariant	when the target of dInitiativeRetiresApplication, RetireDate is not before the initiative's finishDate (warning)
dApplicationFunction		Encapsulates functions supported by applications.
dBusinessArea		an organization of Business interests, can be decomposed.
dBusinessProcess		A sequence of activities that produce a specific service or product supporting a particular business capability

Stereotype	Tagged Value : Type [multiplicity]	Notes
	Category : String [0..1]	A free categorisation of the process, following your own taxonomy, e.g. "Core", "Support", "Management".
	isAdopted : dboolean [0..1]	true when the people who should follow the process actually do; otherwise false.
	isDocumented : dboolean [0..1]	true when the process is written down (procedure, model or work instruction); otherwise false.
	isEffective : dboolean [0..1]	true when the process delivers its expected outcome; otherwise false.
	isOptimized : dboolean [0..1]	true when the process has been improved for cost and speed and has no known waste; otherwise false.
	LastStandardReviewDate : DateTime [0..1]	When the process was last reviewed.
	NextStandardReviewDate : DateTime [0..1]	When the process is next due for review.
	Owner : String [0..1]	The person or role accountable for the process.
	ProcessCriticality : ProcessCriticality [0..1]	How critical the process is to business operations: High, Medium or Low. Absent means not assessed; unknown says the same and is kept for older models.
	ProcessType : dProcessType [0..1]	How the process is carried out: Manual, Automatic (by IT) or Client facing.
	ProcessVolumetrics : dQuantityValue [0..1]	How often the process runs, as a count per period, e.g. 120 {claim} per P1D. Absent means not measured.
	RetireDate : Date [0..1]	The date the process stops, or is planned to stop, being used. Absent means no retirement is planned.
	Source : String [0..1]	Where the information about this process came from, e.g. a document, interview or tool export.
	StandardCreationDate : Date [0..1]	The date the process was first defined or adopted as a standard.
dBusinessService		A repeatable activity; a discrete behavior that a building block may be requested or otherwise triggered to perform. A service provided by Business that achieve a business outcome in response to a request.
	Category : String [0..1]	A free categorisation of the service, following your own taxonomy, e.g. "Customer service", "Internal".
	LastStandardReviewDate : Date [0..1]	When the service was last reviewed.
	NextStandardReviewDate : Date [0..1]	When the service is next due for review.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	Owner : String [0..1]	The person or role accountable for the service.
	RetireDate : Date [0..1]	The date the service stops, or is planned to stop, being offered. Absent means no retirement is planned.
	Source : String [0..1]	Where the information about this service came from, e.g. a document, interview or tool export.
dBusinessUseCase		Encapsulates business processes producing value for an enterprise.
	Extensions : String [0..1]	Alternative and error paths, each referring to the step of the main scenario where it branches, e.g. "3a. Policy has lapsed: ...".
	GoalInContext : String [0..1]	The goal of the primary actor, written as a short verb phrase, e.g. "Enrol student in seminar".
	isCore : Boolean [0..1]	true when the use case is part of the core of the system and must be delivered first; otherwise false.
	Level : String [0..1]	The level of detail of the use case, e.g. summary, user goal or sub-function.
	MainSuccessScenario : String [0..1]	The numbered steps of the normal path through the use case, when everything goes as expected (the "happy path").
	OtherActors : String [0..1]	The secondary actors that take part in the use case, besides the primary actor. Repeats what the diagram shows, so the use case can be read on its own.
	Precondition : String [0..1]	What must be true before the use case can start, e.g. "the customer is logged in" or another use case that must have completed.
	Scope : String [0..1]	The system or business area the use case describes, e.g. "Claims portal".
	Trigger : String [0..1]	The event that starts the use case, e.g. "customer reports a car accident".
dCapability		Represents an organizational capability to achieve business outcomes.Can be current available in an Organization Unit (or) A business-focused outcome that is delivered by the completion of one or more work packages (by increasing the capability Maturity).
	BusinessValue : CapabilityBusinessValue [0..1]	How much value the capability brings to the enterprise: Low, Medium or High, or Support (needed but not distinctive) or Differentiator (sets you apart from competitors).
	Category : String [0..1]	A free categorisation of the capability, following your own taxonomy, e.g. "Customer management".
	CostScope : dCostScope [0..1]	Deprecated: dCapability.Cost is removed. A capability's own cost periods (dCapabilityHasCostPeriod) hold only direct cost, and its full cost is derived from the cost periods of its linked roles and applications. Ignored by tools; do not set.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	Criticality : Int32 [0..1]	How critical the capability is, measured by the cost of it failing, on a scale of 1 (minor) to 100 (the business cannot operate).
	HierarchyLevel : Int32 [0..1]	The level of this capability in the capability map: 1 for top-level capabilities, 2 for their children, and so on. It is not the maturity: use Increments for that.
	Increments : dIncrement [0..1]	The current (as-is) maturity of the capability, from None (0) to Measured (5). Absent means not assessed, which is not the same as None.
	IncrementsToBe : dIncrement [0..1]	The maturity the capability should reach (to-be), from None (0) to Measured (5). Initiatives that raise it are linked by dInitiativeIncreasesMaturityOf. Absent means no target set.
	IncrementSupplyChain : dIncrement [0..1]	The maturity of this capability among your partners and suppliers, from None (0) to Measured (5). Absent means not assessed.
	IncrementVertical : dIncrement [0..1]	The typical maturity of this capability in your industry, from None (0) to Measured (5), for benchmarking. Absent means not assessed.
	Owner : String [0..1]	The person or role accountable for the capability.
	Risk : Int32 [0..1]	How much risk is attached to the capability, on a scale of 1 (low) to 100 (very high).
	Source : String [0..1]	Where the information about this capability came from, e.g. a document, interview or tool export.
	Invariant	its own cost periods (dCapabilityHasCostPeriod) hold only cost that no linked role or application carries
dCluster		Represents a set of nodes working in the same Region, akin to ArchiMate’s Node concept within a server infrastructure.
dContainer		Encapsulates application components and services, mirroring ArchiMate's concept of a Node, where an application runs.
dContract		Provides an agreement between a service consumer and a service provider that establishes functional and non-functional parameters for interaction.
	AvailabilityCharacteristics : String [0..1]	The availability the provider must deliver, e.g. "99.9 % per month".
	billableUnit : String [0..1]	The unit that is billed, as a UCUM code: a usage unit such as {transaction} or GBy for PerUnit, or a party such as {policy} or {subscriber} for PerPartyPerPeriod.
	ConsumingService : String [0..1]	The name of the service that uses this contract. Prefer the contract's relationships when the service is in the model.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	Description : String [0..1]	A short description of what the contract covers. When pricingBasis is Other, explain the pricing here.
	effectiveFrom : Date [0..1]	The first day the contract applies. Absent means it applies from before the model's time frame.
	effectiveTo : Date [0..1]	The last day the contract applies; after effectiveFrom. Absent means open-ended.
	Growth : String [0..1]	The expected growth of demand under the contract, e.g. "+5 % per year".
	Owner : dRole [0..1]	The role accountable for the contract.
	pricingBasis : dPricingBasis [0..1]	What the price is charged for. Read it together with billableUnit and tariff: PerUnit (tariff x units used), PerPartyPerPeriod (tariff per party per tariff.per), FlatFee (tariff per tariff.per), Tiered (volume bands, not modelled) or Other (explained in Description). Absent means not priced.
	ProvidingService : String [0..1]	The name of the service that provides this contract. Prefer the contract's relationships when the service is in the model.
	renewal : Duration [0..1]	How often the contract renews, as an xsd:duration, e.g. P1Y. Absent means it does not renew automatically.
	ResponseRequirements : String [0..1]	The response times the provider must meet, e.g. "95 % of requests within 2 s".
	ServiceTimes : String [0..1]	When the service must be available, e.g. "Mon-Fri 08:00-18:00 ET" or "24x7".
	Source : String [0..1]	Where the information about this contract came from, e.g. the signed agreement.
	tariff : dQuantityValue [0..*]	The price components, each a quantity with its own basis, e.g. 105 CAD per P1M basis "per unit" and 1 CAD per P1M basis "per door". Their sum is the price. Read together with pricingBasis and billableUnit.
	Throughput : String [0..1]	The capacity the provider must support, e.g. "500 requests per second".
	Invariant	effectiveTo is after effectiveFrom when both are set
dController		Encapsulates application behavior and flow logic.
dCostPeriod		Spend, or a spending cap, of one element in one period, planned or actual. Spend belongs to an initiative, application, role, capability, product or resource; a cap belongs to a program (its budget). costNature says whether it is capital or operating spend, and costFunction which line of the income statement an operating cost lands on. Spend can be included in the financial period of the organisation unit whose books it lands in (dFinancialPeriodIncludesCostPeriod).
	amount : dQuantityValue [1]	The money spent, or allowed, in the period, with an ISO 4217 currency as unit. Use kind Actual for spend

Stereotype	Tagged Value : Type [multiplicity]	Notes
		that happened and Estimate for a plan.
	costFunction : dCostFunction [0..1]	For an operating cost, the line of the income statement it lands on: CostOfRevenue, SellingGeneralAdmin or ResearchDevelopment. Not set for Capex, which is not an income-statement cost. Absent means not classified.
	costNature : dCostNature [0..1]	Whether the amount is capital (Capex) or operating (Opex) expenditure. Absent means the amount covers both, as a budget cap does.
	periodLength : Duration [1]	The length of the period, as an xsd:duration, e.g. P1M, P3M, P1Y. The cost periods of one owner must not overlap.
	periodStart : Date [1]	The first day of the period.
	Invariant	amount is money: its unit is an ISO 4217 code
	Invariant	a budget period (dProgramHasBudgetPeriod) is a cap, not spend: it is never included in a financial period
	Invariant	costFunction is not set when costNature is Capex
	Invariant	belongs to exactly one owner: an initiative, program, application, role, capability, product or resource (one of the Has...CostPeriod or HasBudgetPeriod relationships)
dDataEntity		An encapsulation of data that is recognized by a business domain expert as a thing. Logical data entities can be tied to applications, repositories, and services and may be structured according to implementation considerations.
	Category : String [0..1]	A free categorisation of the data entity, following your own taxonomy, e.g. "Master data", "Transaction".
	isAccessible : dboolean [0..1]	true when the people and systems that need the data can get it; otherwise false.
	isAccurate : boolean [0..1]	true when the data correctly represents the real-world thing it describes; otherwise false.
	isTimely : dboolean [0..1]	true when the data is up to date when it is used; otherwise false.
	Owner : String [0..1]	The person or role accountable for the data (the data owner).
	PrivacyClassification : String [0..1]	How restricted access to the data is, following your classification scheme, e.g. Public, Internal, Confidential, Personal.
	RetentionClassification : String [0..1]	How long the data must be kept, following your retention scheme, e.g. "7 years after policy end".

Stereotype	Tagged Value : Type [multiplicity]	Notes
	Source : String [0..1]	Where the information about this data entity came from, e.g. a data model or interview.
dDecision		Represents decisions and their impact on architecture or processes.
	Alternatives : String [0..1]	The options considered, each with a short explanation of its pros and cons.
	Assumptions : String [0..1]	What is believed to be true about the problem and the constraints on the solution when the decision is taken.
	Decision : String [0..1]	The option chosen, stated clearly, with references to related work products if needed.
	Implications : String [0..1]	What follows from the decision: work to do, constraints it adds, and risks it creates.
	Justification : Memo [0..1]	Why this option was chosen, how it complies with the architecture principles, and why any deviation from them is accepted.
	Motivation : String [0..1]	Why the decision matters and why it has to be taken now.
	RelatedDecisions : dDecision [0..*]	Other decisions this one depends on, replaces or affects.
	Subject Area : String [0..1]	The area of concern the decision belongs to, e.g. "Integration", "Data".
	Topic : String [1]	The specific question the decision answers, e.g. "Which message broker for claims events?".
dDeploymentModel		The root class for the description of a Deployments to roll out an application, service, or container to a set of resources. akin to ArchiMate's Technology Layer deployment views.
dDeploymentNode		Represents a physical machine or virtual server that hosts services, applications, or containers, akin to ArchiMate’s Node concept in the Technology Layer.
	ArchitectureType : ArchitectureType [0..1]	The CPU architecture of the node, e.g. AMD64, ARM64.
	CPU : String [0..1]	The number and type of CPUs, e.g. "8 vCPU Intel Xeon".
	Disk : String [0..1]	The disk capacity of the node, with its unit, e.g. "500 GB SSD".
	hostname : String [0..1]	The host name of the node on the network, e.g. claims-db-01.
	IP : String [0..1]	The private IP address of the node, e.g. 10.0.4.12.
	OS : String [0..1]	The operating system and version installed on the node, e.g. "Ubuntu 24.04 LTS".

Stereotype	Tagged Value : Type [multiplicity]	Notes
	Owner : String [0..1]	The person or team responsible for the node.
	PublicIP : String [0..1]	The public IP address of the node, when it has one. Leave empty for nodes reachable only on the private network.
	RAM : String [0..1]	The memory of the node, with its unit, e.g. "32 GB".
dEvent		Something that happens, inside or outside the organization, that triggers behaviour or results from it.
	duration : Duration [0..1]	How long the event lasts, as an xsd:duration, e.g. PT4H. Use it when the dates are not known.
	endDate : DateTime [0..1]	When the event, or the window of a possible event, ends; after startDate. Absent means it has not ended or is instantaneous.
	eventType : dEventType [0..1]	Where the event comes from: Business (a business fact), Temporal (a point in time, e.g. month end), External (outside the organisation) or Technical (a system event). Absent means not classified.
	frequency : dQuantityValue [0..1]	How often a recurring event happens, as a count per period, e.g. 120 {claim} per P1D. Set only when nature is Recurring.
	likelihood : Int32 [0..1]	How likely a possible event is, on a scale of 1 (very unlikely) to 100 (almost certain), as for dRisk.likelihood. Set only when nature is Possible.
	nature : dEventNature [0..1]	Recurring: a designed trigger that happens regularly (set frequency). Possible: an event in a scenario that may happen (set likelihood). Occurred: an event that has happened (set startDate).
	owner : dRole [0..1]	The role accountable for responding to the event.
	Source : String [0..1]	Where the information about this event came from, e.g. an incident record or a scenario workshop.
	startDate : DateTime [0..1]	When an occurred event started, or when the window of a possible event opens.
	Invariant	frequency is set only when nature is Recurring
	Invariant	likelihood is set only when nature is Possible
	Invariant	endDate is after startDate when both are set
dFeature		A Feature is a fact that set his related Requirement to be true
	Author : String [0..1]	Who wrote the feature: name and role in the project, e.g. "Ana Diaz, Analyst".

Stereotype	Tagged Value : Type [multiplicity]	Notes
	isImplemented : dboolean [0..1]	true when the feature has been built and delivered; otherwise false.
	Proofreader : String [0..1]	Who checked the feature: name and role in the project, e.g. "Tom Lee, Client".
dFinancialPeriod		An organisation unit's reported or planned results for one period. It holds the income-statement lines (revenue, costOfRevenue, operatingExpense, depreciation, interestExpense, incomeTax), the cash line capitalExpenditure, and investment figures. Subtotals such as gross margin, EBITDA, EBIT, net income and free cash flow are derived from these lines and are never stored. The cost and revenue periods it includes show where its figures come from. Linked to its unit by dOrganizationUnitReportsFinancialPeriod. Use kind Actual on its amounts for reported results and Estimate for a plan.
	capitalExpenditure : dQuantityValue [0..1]	Capital spend in the period, a cash line that is not part of the income statement; it reaches the income statement later as depreciation. It includes the included cost periods whose costNature is Capex. Same currency as revenue.
	costOfRevenue : dQuantityValue [0..1]	The direct cost of what was sold in the period, e.g. claims incurred for an insurer. It includes the included cost periods whose costFunction is CostOfRevenue. Same currency as revenue.
	depreciation : dQuantityValue [0..1]	Depreciation charged in the period: the share of earlier Capex spread over the assets' useful life. A non-cash cost. Same currency as revenue.
	incomeTax : dQuantityValue [0..1]	Income tax charged in the period. Same currency as revenue.
	interestExpense : dQuantityValue [0..1]	Interest paid or accrued on debt in the period. Same currency as revenue.
	investedAssets : dQuantityValue [0..1]	The value of invested assets at the end of the period; set asOf to that date. Same currency as revenue.
	investmentIncome : dQuantityValue [0..1]	Income from invested assets in the period. Same currency as revenue.
	operatingExpense : dQuantityValue [0..1]	All other operating cost in the period, including revenueLinkedExpense and the included cost periods whose costFunction is SellingGeneralAdmin or ResearchDevelopment. Excludes depreciation. Same currency as revenue.
	periodLength : Duration [1]	The length of the period, as an xsd:duration, e.g. P1Y for a fiscal year, P3M for a quarter. Periods of the same organisation unit must not overlap.
	periodStart : Date [1]	The first day of the period.
	revenue : dQuantityValue [0..1]	Revenue earned in the period, e.g. net earned premium for an insurer, with an ISO 4217 currency as unit. It includes the revenue periods the statement includes (dFinancialPeriodIncludesRevenuePeriod). Use kind Actual for reported results and Estimate for a plan.
	revenueLinkedExpense : dQuantityValue	The part of operatingExpense that grows and shrinks with revenue, e.g. commission and premium tax.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	[0..1]	Same currency as revenue.
	Invariant	all amounts in one period use the same ISO 4217 currency
	Invariant	the cost periods it includes add up, by line, to no more than the statement line they land on: CostOfRevenue to costOfRevenue, SellingGeneralAdmin and ResearchDevelopment to operatingExpense, Capex to capitalExpenditure (warning)
	Invariant	an included cost or revenue period lies within the financial period and uses its currency
	Invariant	the revenue periods it includes add up to no more than revenue (warning)
dGoal		a Goal is a measurable scope that the organization wants to achieve. Can be hierarchically decomposed.
	AssumptionType : dAssumptionType [0..*]	The levels at which assumptions were made to reach the goal, e.g. Business, Technology. The assumptions themselves belong in the simulation scenario, not here.
	Priority : Int32 [0..1]	How important the goal is compared with the others, as a percentage from 1 to 100.
	Value_amount : dQuantityValue [0..1]	The baseline: the level of the targeted KPI when the goal was set, e.g. 103.2 % for the combined ratio. Set asOf to that date. The unit equals the KPI's UnitOfMeasure.
	Value_Goal : dQuantityValue [0..1]	The change the goal aims for, from Value_amount, in the same unit. Negative means reduce, e.g. -6.2 % to take the combined ratio from 103.2 % to 97.0 %. The target level is Value_amount + Value_Goal; zero means hold the level. The deadline is the latest finishDate of the goal's objectives.
	Value_Name : String [0..1]	The name of what the goal changes, e.g. "Combined ratio". It should match the name of the KPI the goal targets.
	Invariant	Value_amount.unit and Value_Goal.unit equal the UnitOfMeasure of the KPI the goal targets (dGoalTargetsdMeasurementIndicator)
	Invariant	a goal with Value_Goal targets exactly one KPI
dGrowthPackage		A set of actions identified to achieve one or more objectives for the business. A work package can be a part of an Initiative, a complete project, or a part of a Program. A Growth pack is a collection of Initiatives architected for consistency and aimed to be executed in the same period.
dInitiative		An activity carried out by a Business Unit or organization to achieve a particular goal or improve the Maturity of a certain capability.
	ability to Implement : Int32 [0..1]	How confident you are that the organisation can deliver the initiative, as a percentage from 1 to 100.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	actualFinishDate : Date [0..1]	The date the initiative actually finished; not before actualStartDate. Absent means not finished.
	actualStartDate : Date [0..1]	The date work actually started. Absent means not started.
	Description : String [0..1]	A one- or two-sentence summary of the initiative, e.g. "Replace PolicyMaster with NovaPAS".
	DetailedDescription : String [0..1]	The full description of the initiative: scope, approach, deliverables and what is out of scope.
	EstimatedTotalCost : dQuantityValue [0..1]	The current estimate of the total cost, with an ISO 4217 currency as unit. When the initiative also has cost periods (dInitiativeHasCostPeriod), their amounts should add up to it.
	finishDate : Date [1]	The planned finish date. Record the real one in actualFinishDate.
	Impacted Capability : dCapability [0..*]	Deprecated: duplicates the relationship dInitiativeIncreasesMaturityOf. Link the initiative to the capabilities it improves with that relationship instead.
	InitiativeDuration : Duration [0..1]	The planned duration, as an xsd:duration, e.g. P18M. When startDate and finishDate are both set, it must equal the time between them.
	initiativeHealth : InitiativeStatus [0..1]	The health of the initiative as a traffic light, from DeepGreen (on track) to Red (in serious trouble).
	kind : String [0..1]	The type of initiative, following your own taxonomy, e.g. "Transformation", "Compliance", "Run".
	purpose : String [0..1]	Why the initiative exists: the problem it solves or the outcome it delivers, in one sentence.
	Rank : String [0..1]	The position of the initiative in the portfolio ranking, e.g. 1 for the most important.
	RelatedProgram : String [0..1]	Deprecated: free text, not a link. Put the initiative in a dGrowthPackage that belongs to a dProgram instead.
	startDate : Date [1]	The planned start date. Record the real one in actualStartDate.
	Invariant	InitiativeDuration must equal finishDate - startDate when all three are set
	Invariant	actualFinishDate is not before actualStartDate
	Invariant	the cost periods of the initiative (dInitiativeHasCostPeriod) do not overlap
	Invariant	the sum of the amounts of its cost periods equals EstimatedTotalCost when both are set (warning)
dIPRange		Defines a range of IP addresses (CIDR) used within a network. This aligns with ArchiMate's concept of Infrastructure Service, providing connectivity.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	CIDR : String [1]	The address range in CIDR notation: an IP address, a slash and the number of network bits, e.g. 192.168.0.0/24 for 256 addresses.
	ipRangeType : ipRangeType [0..1]	What the range is used for: Pod (container pods), Service (cluster services) or subNetwork.
	Public : dboolean [0..1]	true when the addresses in the range are reachable from the internet; otherwise false.
dIssue		Am issue is a concrete problem that is affecting a business. Also, a fact that sets a Requirement to be false. (see also Risk)
	Author : String [0..1]	Who raised the issue: name and role in the project.
	Responsible : String [0..1]	Who is responsible for closing the issue: name and role. Use none until it is assigned.
dJSON_Attribute		Defines attributes in JSON data structures.
	jSONtype : dJSON_Type [1]	The JSON type of the attribute, as a dJSON_Type that defines its type and constraints.
dJSON_Datatype		Represents a data type used within JSON structures, defining the format and constraints of data elements in a schema. JSON Data Types include standard types such as string, number, object, array, boolean, and null, ensuring consistency and validation in data exchange between systems.
dJSON_Element		Represents elements or nodes in JSON schemas.
dJSON_Schema		The Schema Object allows the definition of input and output data types. These types can be objects, but also primitives and arrays. This object is based on the JSON Schema Specification Draft 4 and uses a predefined subset of it. On top of this subset, there are extensions provided by this specification to allow for more complete documentation
	schema : SchemaVersion [1]	The JSON Schema draft the schema follows, as its $schema URI, e.g. http://json-schema.org/draft-07/schema#.
	schemaFileName : String [0..1]	The name of the file the schema is generated to, e.g. claim.schema.json. Absent means the generator chooses it.
dJSON_SchemaSubSet		Represents subsets of JSON schema structures.
	default : String [0..1]	The value to assume when none is given.
	enum : string [0..1]	The allowed values, comma-separated, e.g. open, closed, rejected.
	exclusivemaximum : Boolean [0..1]	true when the value must be strictly less than maximum; otherwise it may equal it.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	exclusiveminimum : Boolean [0..1]	true when the value must be strictly greater than minimum; otherwise it may equal it.
	format : format [0..1]	The format a string value must have, e.g. date, email, uri.
	maximum : Decimal [0..1]	The largest value a number may have.
	maxItems : Int32 [0..1]	The maximum number of items in an array.
	maxlength : Int32 [0..1]	The maximum number of characters of a string value.
	maxProperties : Int32 [0..1]	The maximum number of properties an object may have.
	minimum : Decimal [0..1]	The smallest value a number may have.
	minItems : Int32 [0..1]	The minimum number of items in an array.
	minlength : Int32 [0..1]	The minimum number of characters of a string value.
	minProperties : Int32 [0..1]	The minimum number of properties an object must have.
	multipleof : Decimal [0..1]	A number the value must be a multiple of, e.g. 0.01 for amounts in cents.
	pattern : String [0..1]	A regular expression a string value must match, e.g. ^[A-Z]{3}-[0-9]{4}$.
	required : String [0..1]	The names of the properties an object must have, comma-separated.
	uniqueItems : Boolean [0..1]	true when all items of an array must be different; otherwise false.
dJSON_Type		Represents data type definitions for structured information.
	enum : String [0..1]	The allowed values, comma-separated, e.g. open, closed, rejected.
	exclusivemaximum : Boolean [0..1]	true when the value must be strictly less than maximum; otherwise it may equal it.
	exclusiveminimum : Boolean [0..1]	true when the value must be strictly greater than minimum; otherwise it may equal it.
	format : format [0..1]	The format a string value must have, e.g. date, email, uri.
	maximum : Decimal [0..1]	The largest value a number may have.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	maxlength : Int32 [0..1]	The maximum number of characters of a string value.
	minimum : Decimal [0..1]	The smallest value a number may have.
	minlength : Int32 [0..1]	The minimum number of characters of a string value.
	multipleof : Decimal [0..1]	A number the value must be a multiple of, e.g. 0.01 for amounts in cents.
	pattern : String [0..1]	A regular expression a string value must match, e.g. ^[A-Z]{3}-[0-9]{4}$.
dLocation		A place where business activity takes place and can be hierarchically decomposed.
	AreaCode : String [0..1]	The postal code of the location, e.g. M5V 2T6.
	City : String [0..1]	The city of the location.
	Country : String [0..1]	The country of the location, as an ISO 3166-1 alpha-3 code, e.g. DEU, ITA, CAN.
	EmailID : String [0..1]	The contact e-mail address for the location.
	PhoneNumber : String [0..1]	The contact phone number for the location, in international format, e.g. +1 416 555 0100.
	Province : String [0..1]	The province, state or region of the location.
	Street : String [0..1]	The street address of the location, e.g. "100 King St W".
dLogicalAppComponent		Encapsulates logical software components independent of implementation or technology.
	isMeetingBusinessNeeds : dboolean [0..1]	true when the component meets today's business needs; otherwise false.
	isMeetingFutureNeeds : dboolean [0..1]	true when the component can meet the business needs foreseen in the target architecture; otherwise false.
	isUsable : dboolean [0..1]	true when users find the component usable for their work; otherwise false.
dMeasurementArea		Represents areas of organizational or business performance measurement. The organization or Business units performance objective is captured as measurement area.
	Definition : String [0..1]	What the area measures, e.g. "Financial performance", and why it matters to the organisation.
dMeasurementCategory		Categorizes measurements within a measurement area.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	Definition : String [0..1]	What the category measures within its area, e.g. "Underwriting profitability".
	MeasurementArea : dMeasurementArea [1]	The measurement area this category belongs to.
dMeasurementGrouping		a groups of Measurement Indicator that can be aggregated into Categories
	Definition : String [0..1]	What the KPIs in this grouping have in common, e.g. "Loss and expense ratios".
	MeasurementCategory : dMeasurementCategory [1]	The measurement category this grouping belongs to.
dMeasurementIndicator		The specific measure captured for a Organization or Business Unit which can be measured and quantifiable. A measurement is a Key Performance indicator
	CurrentLevel : dQuantityValue [0..1]	The last measured value of the KPI, in UnitOfMeasure, with asOf set to the measurement date.
	Definition : String [0..1]	How the KPI is calculated and what it means, e.g. "(claims incurred + expenses) / net earned premium".
	LastStandardReviewDate : DateTime [0..1]	When the KPI's definition was last reviewed.
	SatisfactionLevel : dQuantityValue [0..1]	The standing threshold at which the KPI is considered satisfactory, in UnitOfMeasure. A goal's target is set on the goal (Value_amount + Value_Goal), not here.
	UnitOfMeasure : String [0..1]	The unit of the KPI, as an ISO 4217 currency or a UCUM code, e.g. %, CAD, d. CurrentLevel, and the values of any goal that targets the KPI, must use this unit. Absent means the KPI has no unit.
	Invariant	CurrentLevel.unit must equal UnitOfMeasure
dModelClass		Represent a class of the domain model, that can be converted into a Persistent domain class, must inherit from Node. It is the application's dynamic data structure
	_metatype : String [1]	The generator's type for this class. Keep the default, Model Class.
	child_order : String [0..1]	The order of the associated child classes, separated by |, e.g. for Recipe: Image|Info|AdminInfo.
	display_value : String [0..1]	The values shown to identify an instance, separated by |, e.g. for Recipe: name|AdminInfo/status.
	initparams : String [0..1]	The configuration section used to initialise the class's data mapper, e.g. database.
	is_searchable : boolean [0..1]	true when instances are included in the default search; otherwise false.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	is_soap : boolean [0..1]	true when a service interface (SOAP or REST) is generated for the class; otherwise false.
	orderby : String [0..1]	The default sort order: none, sortkey (adds a column for explicit ordering) or the name of a dValue optionally followed by ASC or DESC, e.g. "name ASC".
	parent_order : String [0..1]	The order of the associated parent classes, separated by |.
	pk_name : String [0..1]	The name of the primary key column, e.g. id. The generator adds it if no attribute has that name.
	table_name : String [0..1]	The name of the database table. Empty means the class name is used.
dNetwork		The main communication medium between services and deployments, mapped to ArchiMate Communication Networks, facilitating inter-service communication.
	NetworkQuality : String [0..1]	The quality of the network connection: High, Medium or Low.
	NetworkType : NetworkTypes [0..1]	The kind of network: LAN, WAN or Mesh.
dObject		an Object is an instance of a class in a particular moment in runtime that can have its own state and data values.
dObjective		The Objective is a non material achievement, other than a Goal, the objective has no specific value to be measured. it's a time-bounded milestone for an organization used to demonstrate progress towards a goal.
	Category : String [0..1]	A free categorisation of the objective, following your own taxonomy.
	finishDate : Date [1]	The date by which the objective must be reached. The goal's deadline is the latest finishDate among its objectives.
	Owner : String [0..1]	The person or role accountable for reaching the objective.
	Source : String [0..1]	Where the information about this objective came from, e.g. the strategic plan.
	Value_Goal : dQuantityValue [0..1]	The change from the goal's Value_amount to reach by finishDate, in the goal's unit, e.g. -3.0 % by the first milestone. Absent means only the last objective carries the goal's target.
dOnPremise		an organization of Business interests
dOpinion		A view or judgment formed about a Topic. That is formulated in an explicit fashion. Captures explicit stakeholder judgments on a topic.
dOpinionInner		Captures implicit stakeholder opinions related to a topic.

Stereotype	Tagged Value : Type [multiplicity]	Notes
		A Stakeholder’s opinion that is relevant for the topic but is not formulated explicitly.
dOrganizationUnit		Defines organizational units responsible for roles, can be decomposed
	HeadCount : dQuantityValue [0..1]	The number of people working in the unit, with unit {person}. Can be derived by adding up the FTEs of the unit's roles. Absent means not counted.
	Invariant	the financial periods the unit reports (dOrganizationUnitReportsFinancialPeriod) do not overlap
dPhysicalService		a technical implementation of a service that runs on physical or virtual infrastructure, providing specific functionalities, usually in the form of software applications, middleware, databases, or platforms. It is concerned with the deployment, operation, and management of the technology stack, ensuring that the service is available, performant, and reliable. Examples include web servers, APIs, databases, and network services.
	Category : String [0..1]	A free categorisation of the service, following your own taxonomy, e.g. "Database", "Messaging".
	execution_mode : executionMode [0..1]	Whether callers wait for the result (synchronous) or not (asynchronous).
	LastStandardReviewDate : Date [0..1]	When the service was last reviewed against the technology standards.
	NextStandardReviewDate : Date [0..1]	When the service is next due for review against the technology standards.
	Owner : String [0..1]	The person or team responsible for running the service.
	port : String [0..1]	The network port the service listens on, e.g. 5432.
	protocol : ProtocolTypes [0..1]	The protocol the service uses on that port, e.g. https, TCP, UDP.
	RetireDate : Date [0..1]	The date the service is, or is planned to be, switched off. Absent means no retirement is planned.
	Source : String [0..1]	Where the information about this service came from, e.g. a CMDB export.
dPrinciple		Defines qualitative statements guiding architecture intent.
	Implications : String [0..1]	What the principle means in practice: what people must do, or stop doing, to follow it.
	Owner : dStakeholder [0..1]	The stakeholder who owns the principle and decides on exceptions.
	PrincipleID : String [0..1]	Unique identifier of the principle, e.g. PR-DATA-01.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	PrincipleMeasurement : dMeasurementIndicator [0..1]	The KPI used to check whether the principle is followed. Absent means it is not measured.
	Priority : String [0..1]	The priority of the principle relative to the others, e.g. 1 for the highest, used when principles conflict.
	Rationale : String [0..1]	Why the principle is needed and the outcome it leads to.
	Source : String [0..1]	Where the principle came from, e.g. the architecture board minutes of a given date.
	Statement : String [1]	The principle itself, in one clear sentence, e.g. "Data is an asset".
	Type : dPrincipleType [0..1]	The domain of the principle: guiding, business, data, application, integration or technology.
dProduct		Encapsulates outputs produced by business processes.
	Owner : dRole [0..1]	The role accountable for the product.
	Price : dQuantityValue [0..1]	The list price of the product, with an ISO 4217 currency as unit, e.g. 1200 CAD per P1Y for an annual premium. Contracts can set their own tariff. Absent means not priced.
	RetireDate : Date [0..1]	The date the product is, or is planned to be, withdrawn from sale. Absent means no withdrawal is planned.
	Source : String [0..1]	Where the information about this product came from, e.g. the product catalogue.
dProgram		A group of Growth Packages that achieve a complex outcome for the organization
	Invariant	the budget periods of the program (dProgramHasBudgetPeriod) do not overlap
dPublicCluster		A public cloud-based cluster, following the ArchiMate concept of Infrastructure Service provided by external cloud providers (Google, AWS, etc.).
	configFile : String [0..1]	The name of the configuration file used to create or connect to the cluster, e.g. config_google.conf.
	NodeNum : String [0..1]	The number of nodes in the cluster, e.g. 3.
	provider : dCloudProvider [1]	The cloud provider that runs the cluster, e.g. AWS, Azure, Google.
dQuantityValue		Represents a measurable value expressed as a numeric amount and a unit of measurement. It provides a reusable structure for quantitative properties of DAF elements.
	amount : Decimal [1]	The numeric value, as a decimal, e.g. 0.02. Its meaning is given by unit, per and basis.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	asOf : Date [0..1]	The date the value refers to, e.g. the date of a measurement or the end of a period. Absent means not dated.
	basis : String [0..1]	What amount is counted for, when it is a ratio to something other than time, e.g. "per FTE", "per claim". Absent means no such denominator.
	kind : dQuantityKind [0..1]	How the value was obtained: Actual (measured or reported), Estimate (planned or forecast) or Assumption (assumed for a scenario). Absent means not known; unspecified says the same and is kept for older models.
	per : Duration [0..1]	The time period amount refers to, as an xsd:duration, for rates and periodic costs: P stands for period and T marks the time part, e.g. PT30M (30 minutes), P1D (1 day), P1M (1 month), P1Y (1 year). Absent means amount is not a rate.
	unit : String [1]	The unit of amount: an ISO 4217 currency code for money, e.g. CAD, otherwise a UCUM code, e.g. %, h, {FTE}, {claim}.
	Invariant	unit must be a valid ISO 4217 or UCUM code
dRegion		A geographic area containing clusters and zones, aligned with ArchiMate's Location concept for representing deployment geography.
dRequirement		A Business guide line about the Enterprise or the project. Requirements can be decomposed. A requirement is formulated in a SMART fashion and uses Moscow verbs (Must Have, Should Have, Could Have, and Would Like to Have.). Must Have: These are the requirements that are considered essential for the success of the initiative and must be meet for the system to function. They are often referred to as "hard" or "mandatory". requirements and are typically expressed in Boolean format as "true" or "1". Should Have: These are the requirements that are considered important, but not essential. They are typically expressed in Boolean format as "maybe" or "0/1" and are often used to prioritize the development of the system. Could Have: These are the requirements that are considered desirable but not essential. They are typically expressed in Boolean format as "false" or "0". They are used to identify potential enhancements to the system that can be added at a later date. Would Like to Have: These are the requirements that are considered less important and are often used to identify potential future enhancements to the system. In addition, the requirement should be SMART, which stands for Specific, Measurable, Achievable, Relevant, and Time-bound. SMART requirements are clear, concise, and easy to understand, and they provide a clear way to measure the success of the project.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	Author : String [0..1]	Who wrote the requirement: name and role in the project, e.g. "Ana Diaz, Analyst".
	Priority : Int32 [0..1]	The priority of the requirement as a percentage from 1 to 100. Requirements are ordered by it.
	Proofreader : String [0..1]	Who confirmed the requirement: name and role in the project, e.g. "Tom Lee, Client". Every requirement needs one.
	Status : String [0..1]	Where the requirement is in the workflow, e.g. Proposed, Validated, Mandatory.
	Type : dRequirementType [0..1]	The kind of requirement, e.g. Business Rule, Security, Performance, Scalability.
dResource		a type of Asset supporting a Business capability
	# of items : Float [0..1]	How many units of the resource are available, e.g. 40 for 40 vehicles.
	depreciationMethod : dDepreciationMethod [0..1]	How its Capex is spread over usefulLife: StraightLine or DecliningBalance. The depreciation schedule is derived from the Capex cost periods, inServiceDate, usefulLife and this method; it is not stored.
	inServiceDate : Date [0..1]	The date the asset went into use; depreciation starts here. Absent means not recorded.
	usefulLife : Duration [0..1]	How long the asset is depreciated over, as an xsd:duration, e.g. P5Y. Absent means the asset is not capitalised or the life is not recorded.
dRevenuePeriod		Revenue earned by one product or contract in one period, reported (kind Actual) or planned (kind Estimate). It can be included in the financial period of the organisation unit whose books it lands in (dFinancialPeriodIncludesRevenuePeriod).
	amount : dQuantityValue [1]	The revenue earned in the period, with an ISO 4217 currency as unit. Use kind Actual for reported revenue and Estimate for a plan.
	periodLength : Duration [1]	The length of the period, as an xsd:duration, e.g. P1M, P1Y. The revenue periods of one product or contract must not overlap.
	periodStart : Date [1]	The first day of the period.
	Invariant	amount is money: its unit is an ISO 4217 code
	Invariant	belongs to exactly one product or contract (dProductHasRevenuePeriod, dContractHasRevenuePeriod)
	Invariant	the revenue periods of one product or contract do not overlap
dRisk		a risk Identifies and assesses potential issue impacting other elements. It is a potential future Issue. Can be

Stereotype	Tagged Value : Type [multiplicity]	Notes
		hierarchically decomposed.
	likelihood : Int32 [0..1]	The probability that the risk becomes an issue, on a scale of 1 (very unlikely) to 100 (almost certain). Absent means not assessed.
	RiskStrategy : dRiskStrategy [0..1]	How the risk is handled: Terminate (remove the cause), Treat (reduce it), Tolerate (accept it) or Transfer (e.g. insure or outsource it).
	Severity : Int32 [0..1]	The impact if the risk happens, on a scale of 1 (negligible) to 100 (critical). Absent means not assessed.
dRole		The usual or expected function of an actor, or the part somebody or something plays in a particular action or event. An actor may have a number of roles.
	CostRate : dQuantityValue [0..1]	The cost of one unit of the role's capacity, e.g. 85000 CAD per P1Y with basis "per FTE". It is a rate, not spend: the spend on the role goes into its cost periods (dRoleHasCostPeriod). Absent means not known.
	DegreeOfUtilisation : dQuantityValue [0..1]	The share of the role's capacity that is in use, with unit %, e.g. 88 %. Absent means not measured.
	FTEs : dQuantityValue [0..1]	The number of full-time equivalents working in the role, with unit {FTE}, e.g. 42 {FTE}. Absent means not counted.
	isSkillsAvailable : dboolean [0..1]	true when the people in the role have the skills it needs; otherwise false.
	isSkillsDefined : dboolean [0..1]	true when the skills the role needs are written down; otherwise false.
	Owner : String [0..1]	The person or unit accountable for the role, e.g. its line manager.
	Source : String [0..1]	Where the information about this role came from, e.g. the HR system.
	User : String [0..1]	A free categorisation of the role, following your own taxonomy.
dRule		A precise business condition or constraint, implemented at a specific point in an application. Modeled as an operation with package visibility on a dController, which acts as the rule set. Its Statement is the human-readable rule passed to an LLM to generate the implementation.
	RuleKind : dRuleKind [0..1]	The OMG category of the rule: Production, Integrity, Derivation or Reaction.
	Statement : String [0..1]	Human-readable statement of the rule, precise enough to pass to an LLM for implementation, e.g. "Ensure Customer.Age >= 18 before Account.Creation".
	SourceEvidence : String [0..1]	Where the rule was found in legacy code: program, paragraph or section, and line range, e.g. "LGICUS01 / 2000-VALIDATE-AGE / L412-L431".

Stereotype	Tagged Value : Type [multiplicity]	Notes
	Certainty : dRuleCertainty [0..1]	Explicit when modeled directly; Inferred when extracted from legacy code.
dSecurityGroup		Controls and manages access and permissions between services, in line with TOGAF's Security Architecture for protecting digital assets.
dSkill		an ability that some role possesses with various degree
	SkillCategory : String [0..1]	The group the skill belongs to, e.g. "Technical", "Domain", "Leadership".
	SkillLevel : String [0..1]	How well the skill is mastered, on your own scale, e.g. Basic, Proficient, Expert.
dStakeholder		Represents any party or force influencing the domain of interest. Any person or force that have an interest in the considered domain. This is typically indicated by name, can be connected with a Role in the organization. A stakeholder has open and Inner opinions that are relevant for the subject matter.
	Attitude : dScala [0..1]	How favourable the stakeholder is to the change: Low (opposed), Medium (neutral) or High (supportive).
	Knowledge : dScala [0..1]	How much the stakeholder knows about the subject: Low, Medium or High.
	Legitimacy : dScala [0..1]	How widely the stakeholder's claim is accepted as appropriate by others: Low, Medium or High.
	Power : dScala [0..1]	How far the stakeholder can impose their will on the outcome: Low, Medium or High.
	RACI : String [0..1]	The stakeholder's responsibility: R (Responsible: does the work), A (Accountable: answers for the result; only one per task), C (Consulted: gives input before the work is done) or I (Informed: is told the outcome).
	role : dRole [0..1]	The role the stakeholder plays in the organisation, when it is in the model.
	Urgency : dScala [0..1]	How much the stakeholder's concerns call for immediate attention: Low, Medium or High.
dSubNetwork		A subset of a network, used to organize and isolate portions of the network. Equivalent to an ArchiMate Node, responsible for networking resources.
dSystem		A system configuration. Settings are modeled as attributes with default values, each attribute of the System Config becomes a line in the configuration file. e.g. SystemConfig .path = c:/users/app
	config : String [1]	The configuration file the settings are written to, e.g. config.ini.
	platform : String [0..1]	The platform the settings apply to, e.g. "production", "Windows".
dTable		Represents physical tables in a database schema.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	Database : String [0..1]	The database schema that holds the table, e.g. claims.
dTest		Represents a mechanism for verifying or validating the functionality, behavior, or compliance of am application component . Tests can include automated or manual procedures designed to ensure alignment with requirements, detect defects, and confirm expected outcomes.
dUserStory		Captures user-centric requirements expressed as stories.
	EstimatedEffort : String [0..1]	The estimated effort to deliver the story, in hours, e.g. 8.
	StoryDetails : Story [1]	The story itself: "As a I want so that ".
	TestScenario : String [0..1]	How to check that the story is done: the situation and the expected result, e.g. "Given ... when ... then ...".
dValue		An attribute of a Node value type used in DAF. These values are persistent
	app_data_type : String [0..1]	How the value is handled by the application: DATATYPE_ATTRIBUTE, DATATYPE_ELEMENT, DATATYPE_DONTCARE or DATATYPE_IGNORE.
	column_name : String [0..1]	The name of the database column. Empty means the value's name is used.
	db_data_type : String [0..1]	The column type in the database, e.g. VARCHAR(255), INT, DATE.
	display_type : String [0..1]	How the value is shown, e.g. text, image.
	input_type : String [0..1]	The HTML input control used to edit the value, e.g. text, or select#fix:key1[val1]|key2[val2] for a fixed list.
	is_editable : boolean [0..1]	true when users can change the value; otherwise false.
	restrictions_description : String [0..1]	The message shown to users when the value breaks a restriction, e.g. "Enter a date as DD.MM.YYYY".
	restrictions_match : String [0..1]	A regular expression the value must match, e.g. [0-3][0-9]\.[0-1][0-9]\.[0-9]{4} for a date.
	restrictions_not_match : String [0..1]	A regular expression the value must not match.
dValueRef		A Node value type used in DAF that reference another Node Value, similar to a foreign Key. These values are persistent
	reference_value : dValue [1]	The dValue this reference points to, like a foreign key.
dValueStream		A representation of an end-to end collection of value-adding Business Processes that create an overall

Stereotype	Tagged Value : Type [multiplicity]	Notes
		result for a customer, stakeholder or end user. A sequence of activities an enterprise undertakes to deliver on a customer request. More broadly, the sequence of activities required to design, produce, and deliver a good or service to a customer, and it includes the dual flows of information and material.
	Criticality : ValueStreamCriticality [0..1]	How much the value stream matters to the customer and the business, from Negligible Impact to Significant Impact. Leave Not Analyzed until it is assessed.
	entrance criteria : String [0..1]	What starts the value stream, e.g. "customer requests a quote".
	exit criteria : String [0..1]	What ends the value stream, i.e. the value delivered, e.g. "policy issued and paid".
	is Decomposed : Boolean [0..1]	true when the value stream has been broken down into stages; otherwise false.
dView		A View used in DAF. Controller, Views and Associations define the application flow
dVolume		Refers to the storage assigned to containers, ensuring persistence, aligning with the Data Object in ArchiMate’s Application Layer.
	MountPath : String [1]	The path inside the container where the volume is mounted, e.g. /var/lib/postgresql/data.
	name : String [0..1]	The name of the volume, e.g. claims-data.
	subPath : String [0..1]	A sub-directory of the volume to mount instead of its root. Empty means the whole volume.
dZone		Defines macro divisions containing regions mapped to ArchiMate’s Infrastructure Service.

8.2 Enumerations

DAF 7.0.0 Beta 6 defines 44 enumerations. They type the tagged values listed above.

Enumeration	Values	Used by
ActorTypes	Active Business Partner, Business Partner, External Worker, Internal Worker, Not Set, Passive Business Partner, Worker	dActor.ActorType
ArchitectureType	AMD64, ARM64, RISC-V, ppc64le, s390x, x86	dDeploymentNode.ArchitectureType
CapabilityBusinessValue	Differentiator, High, Low, Medium, Support	dCapability.BusinessValue
dApplicationStrategy	Refactor, Rehost, Replatform, Retain, Retire, Rewrite	dApplicationComponent.InvestmentStrategy
dApplicationType	COTS, Core, Custom, DB, Instance, Module, Web Site	dApplicationComponent.ApplicationType
dAssumptionType	Application, Business, Data, Enterprise, System, Technology	dGoal.AssumptionType
dBaseStatus	Approved, Modified, Proposed, Rejected, Released, Solved, Validated	not used by a property
dboolean	false, true	dBusinessProcess.isAdopted, dBusinessProcess.isDocumented, dBusinessProcess.isEffective, dBusinessProcess.isOptimized, dDataEntity.isAccessible, dDataEntity.isTimely, dFeature.isImplemented, dIPRange.Public, dLogicalAppComponent.isMeetingBusinessNeeds, dLogicalAppComponent.isMeetingFutureNeeds, dLogicalAppComponent.isUsable, dRole.isSkillsAvailable, dRole.isSkillsDefined

Enumeration	Values	Used by
dCloudProvider	AWS, Azure, DigitalOcean, Google, Other	dPublicCluster.provider
dConstraintType	Invariant, Post-condition, Pre-condition, Process	not used by a property
dCostFunction	CostOfRevenue, ResearchDevelopment, SellingGeneralAdmin	dCostPeriod.costFunction
dCostNature	Capex, Opex	dCostPeriod.costNature
dCostScope	Aggregated, Direct	dCapability.CostScope
dDepreciationMethod	DecliningBalance, StraightLine	dApplicationComponent.depreciationMethod, dResource.depreciationMethod
dEventNature	Occurred, Possible, Recurring	dEvent.nature
dEventType	Business, External, Technical, Temporal	dEvent.eventType
dFeatureStatus	Implemented, Proposed, Rejected, Validated	not used by a property
dIncrement	Defined, Initial, Managed, Measured, None, UnderDevelopment	dCapability.Increments, dCapability.IncrementsToBe, dCapability.IncrementSupplyChain, dCapability.IncrementVertical
dLifecycleStatus	In Development, Live, Phasing Out,	not used by a property

Enumeration	Values	Used by
	Proposed, Retired	
dPricingBasis	FlatFee, Other, PerPartyPerPeriod, PerUnit, Tiered	dContract.pricingBasis
dPrincipleType	Application Principle, Business Principle, Data Principle, Guiding Principle, Integration Principle, Technology Principle	dPrinciple.Type
dProcessType	Automatic, Client facing, Manual	dBusinessProcess.ProcessType
dQuantityKind	Actual, Assumption, Estimate, unspecified	dQuantityValue.kind
dRequirementStatus	Mandatory, Proposed, Validated	not used by a property
dRequirementType	Business Rule, Performance, Persistence, Scalability, Security, Transport	dRequirement.Type
dRiskStrategy	Terminate, Tolerate, Transfer, Treat	dRisk.RiskStrategy
dRuleCertainty	Explicit, Inferred	dRule.Certainty
dRuleKind	Derivation, Integrity, Production, Reaction	dRule.RuleKind
dSatisfactionType	Highly Dissatisfied, Highly Satisfied, Moderately Satisfied, Not Satisfied,	not used by a property

Enumeration	Values	Used by
	Satisfied	
dScala	High, Low, Medium	dStakeholder.Attitude, dStakeholder.Knowledge, dStakeholder.Legitimacy, dStakeholder.Power, dStakeholder.Urgency
executionMode	asynchronous, synchronous	dPhysicalService.execution_mode
format	<none>, date, date-time, email, hostname, idn-email, idn-hostname, ipv4, ipv6, iri, iri-reference, json-pointer, regex, relative-json-pointer, time, uri, uri-reference, uri-template	dJSON_SchemaSubSet.format, dJSON_Type.format
HTTPMethod	delete, get, head, options, patch, post, put, trace	dAPIOperation.hTTPMethod
InitiativeStatus	DeepGreen, Green, Orange, Red, Yellow	dInitiative.initiativeHealth
ipRangeType	Pod, Service, subNetwork	dIPRange.ipRangeType
MimeTypes	application/json, application/octet-stream, application/pdf, application/problem+json, application/x-www-form-urlencoded, application/xml, application/yaml, multipart/form-data, text/csv, text/event-stream, text/html, text/plain	dAPI.consumes, dAPI.produces
NetworkQuality	High, Low, Medium	not used by a property

Enumeration	Values	Used by
NetworkTypes	LAN, Mesh, WAN	dNetwork.NetworkType
ProcessCriticality	High, Low, Medium, unknown	dBusinessProcess.ProcessCriticality
ProtocolTypes	All, RNS, TCP, UDP, http, https	dPhysicalService.protocol
QualityNetwork	High, Low, Medium	not used by a property
SchemaVersion	http://json-schema.org/draft-04/schema#, http://json-schema.org/draft-05/schema#, http://json-schema.org/draft-06/schema#, http://json-schema.org/draft-07/schema#	dJSON_Schema.schema
SchemeType	http, https, ws, wss	dAPI.schemes
ValueStreamCriticality	Limited Impact, Moderate Impact, Negligible Impact, Not Analyzed, Noticeable Impact, Significant Impact	dValueStream.Criticality

8.3 Model Checks

Model checks are implemented using Bellekens EA Validator. See Enterprise Architect Toolpack releases

The following 84 checks are implemented in Checks/checks. CIM, PIM and PSM follow the MDA levels; IDs are unique within a category.

Category / Type	Check (ID and description)	Rationale
All		
	001 An Element with the status not set	A model element must have the status set
	002 Elements still in status Proposed must be reviewed	Proposed status indicates that the Element is probably not managed
	003 The main DAF diagram without objects	No diagram should be empty.
	004 a Diagram must not contain more than 25 elements	Ideally a diagram should not contains more that seven elements
	005 a Diagram should not contain less than 2 elements	Ideally a diagram should contains at least Two elements
	006 Element is not displayed in a diagram	an element that is not deleted must be displayed in a least a diagram
CIM		
Application Architecture	001 A “LogicalAppComponent” should exposes a “BusinessService”	a 'dLogicalAppComponent ' element should be connected to a 'dBusinessService' element by a ' exposes Biz Service' Connector
Business Architecture	001 Goal is Operationalized By Capability	a 'dCapability' element should be connected to a ' dGoal' element by a 'isOperationalizedBy' Connector
	002 Capability is Measured By Measurement Indicator	a 'dCapability' element should be connected to a 'dMeasurementIndicator' element by a 'isMeasuredByf' Connector
	003 Logical Component Supports Capability	a 'dCapability' element should be connected to a 'dLogicalAppComponent' element by a 'Supports Capability' Connector
	004 Business Process Ensures the correct Operation of Capability	a 'dCapability' element should be connected to a 'dBusinessProcess' element by a 'Ensure the correct Operation' Connector
	006 Role executes Capability	a 'dCapability' element should be connected to a Role element
	007 Data Entity informs Capability	a 'dCapability' element should be connected to a Data Entity element
	008 Initiative Increases Maturity Of Capability	a 'dCapability' element can be connected to am Initiative element
	09 Biz Area without Capability	Am area aggregatyes business capabilities
	10 Resource without Capability	A logical application should support the business capability

Category / Type	Check (ID and description)	Rationale
Requirement Management	001 Goal related to Requirement	a 'dRequirement' element should be connected to a 'dGoal' element by a 'is related to' Connector
	002 Requirement isSatisfiedBy Feature	a 'dFeature' element should be connected to a ' dRequirement' element by a 'isSatisfiedBy' Connector
	003 Goal must relate to Requirement or another goal	a Goal element should be connected to a 'Goal' or a requirement
	004 Requirement is connected with another requirement OR a feature	a Goal element should be connected to a 'Goal' or a requirement
	005 Requirement cannot be connected Feature and Issue	a 'dFeature' element should be connected to a ' dRequirement' element by a 'isSatisfiedBy' Connector
Technology Architecture	001 Network Contains DeploymentNode	a 'dNetwork' element should be connected to a 'dDeploymentNode' element by a 'NetworkContainsNode' Connector
	002 Location Hosts DeploymentNode	a 'dLocation' element should be connected to a 'dDeploymentNode' element by a 'Hosts Node' Connector
PIM		
Application Architecture	001 Function has no documentation	Function has no documentation
	002 ApplicationComponent documentation contains long dash '–' character	dApplicationComponent documentation contains long dash '–' character
	003 Application Component does not have documentation	ApplicationComponent does not have documentation
	004 dApplicationComponent name contains 2 spaces between words	dApplicationComponent name contains 2 spaces between words
	005 dApplicationComponent name contains leading or trailing space characters	dApplicationComponent name contains leading or trailing space characters
	006 dApplicationComponent documentation is in HTML format (rather than plain text)	dApplicationComponentdocumentation is in HTML format (rather than plain text)
	007 Application components has no name	Application requires a name
	008 Function has no name	Function requires a name
	009 a function should be associated with an Application	an Application Function must be aggregated by an Application Component (dApplicationAggregatesFunction)

Category / Type	Check (ID and description)	Rationale
	010 ApplicationComponent Aggregates ApplicationFunction	a 'dApplicationComponent' element should be connected to a 'dApplicationFunction' element by a 'dApplicationAggregatesFunction' Connector
	013 Component Realizes Logical Component	a 'dApplicationComponent' element should be connected to a ' dLogicalAppComponent' element by a 'Realizes' Connector
	015 System configuring an Application Component	a 'dApplicationComponent' element should be configured by a 'dSystem' element by a 'configured by' Connector
	016 ApplicationComponent Exposes PhysicalService	a 'dApplicationComponent' element should be connected to a 'dPhysicalService' element by a 'Exposes Service' Connector
	017 PhysicalService Implements BusinessService	a 'dPhysicalService' element should be connected to a 'dBusinessService' element by a 'Implements' Connector
	021 Controller Implements Function	a 'dController' element should be connected to a 'dPhysicalService' element by a 'Implements Service' Connector
	022 Controller Controlls ModelClass	a 'dController' element should be connected to a 'dModelClass' element by a 'Controlls' Connector
	023 Controller should Implementa BusinessUseCase	a 'dController' element should be connected to a 'dBusinessUseCase' element by a 'isImplementedBy' Connector
	024 Controller Governs View	a 'dController' element should be connected to a 'dView' element by a 'Governs' Connector
Business Architecture	001 Business Use Case is is realizing Feature	a BusinessUseCase' element should be connected to a 'dFeature' element by a 'is realized by Use Case' Connector
	002 BusinessService isDescribedBy BusinessUseCase	a 'dBusinessUseCase' element should be connected to a 'dBusinessService' element by a 'isDescribedBy' Connector
	003 Use Cases that are not in exactly one Boundary	A UseCase must appear in exactly 1 Boundary (of all Use Case-diagrams)
Technology Architecture	001 ApplicationComponent isPhysically Hosted By Deployment Node	a 'dDeploymentNode' element should be connected to a 'dApplicationComponent' element by a 'isPhysicallyHostedBy' Connector
PSM		
Data Architecture	000 Class diagrams with more than 25 elements	A Class diagram should have max. 25 elements

Category / Type	Check (ID and description)	Rationale
	001 Unreferenced Enumerations	Search for unnecessary Enumerations
	002 Unreferenced Datatypes	zll the ata type must be described in a class
	003 Enumerations without values	An Enumeration must have at least 1 value.
	004 Model class has no name (or empty name)	Entity has no name (or empty name)
	005 Association class related to other class(es)	
	006 Attribute documentation is in HTML format	HTML format can generate issues
	007 Association documentation contains long dash '–' character	Association documentation contains long dash '–' character
	008 Association documentation is in HTML format (rather than plain text)	Association documentation is in HTML format (rather than plain text)
	009 Attribute documentation contains long dash '–' character	Add documentation
	010 Datatype/Enumeration documentation contains long dash '–' character	Datatype/Enumeration documentation contains long dash '–' character
	011 Datatype/Enumeration documentation is in HTML format (rather than plain text)	Datatype/Enumeration documentation is in HTML format (rather than plain text)
	012 Datatype/Enumeration does not have documentation	Datatype/Enumeration does not have documentation
	013 Classes containing Attributes without datatype	Every Attribute of a Class must have a Datatype
	014 Two classes with the same name in the model	classes with the same name can be confusing
	015 Mode Class without any relations to other entities	Entities without any relations to other entities. This validation skips entities which are marked as >
	016 Model Class does not have Documentation	Entity does not have Documentation
	017 Model Class is directly derived from more than one base class	Entity is directly derived from more than one base class
	018 Entities (classes or association classes) which do not appear in any diagram in the model	Entities (classes or association classes) which do not appear in any diagram in the model

Category / Type	Check (ID and description)	Rationale
	019 Attribute is not defined as public	ensure that the attribute need to be != public
	020 Attribute has no name (or empty name)	Attribute must have a name
	021 Attribute of an entity does not have a type	Attribute of an entity does not have a type
	022 Unnamed association (between 2 model Classes)	all associations must have a name
	023 Unnamed self-association	Unnamed self-association
	024 Association name contains space characters	Association name cannot contains spaces
	025 Duplicate associations (association with exactly the same name)	associations must have unique names
	026 Attribute has no documentation	All Attribute should have a Description.
	027 association should specify navigability	unspecified navigability not recommended for information model
Implementation	001 Initiative IncreasesMaturityOf Capability	a 'dInitiative' element should be connected to a 'dCapability' element by a 'IncreasesMaturityOf' Connector
	002 Decision are attached to Initiative	a Decision element should be connected to a Initiative element by a 'IncreasesMaturityOf' Connector
Technology	001 API does not have base path	Base path is required
	002 API does not have Documentation	Explain the purpose of the API
	003 Operation is not defined as public	operations that are set as Package or Private have special meaning
	004 APIoperation has no documentation	non documented Operations are confusing
	005 API Operation with no TAG	Tag is necessary
	005 API Operation with no relative path	relative path is necessary
	010 API does not have contact	contact is required

Category / Type	Check (ID and description)	Rationale
	011 API does not have license	license is required
	012 API does not have host	host is required

8.4 DAF COLORS

The metamodel uses certain colors to indicate domain of architecture

8.4.1 RGB Colors

- DAF Generic blue: 107,203,236
- Enterprise ARCHITECTURE Green: 155,187,89
- SOLUTION ARCHITECTURE dark blue: 79,129,189
- DESIGN ARCHITECTURE **Dark Orange: 233,191,149**
- Performance **violet: 234,198,234**
- Requirements **red: 222,103,103**
- Stakeholder **light blue: 240,255,255**
- Technology **Green: 156,204,84**
- Strategy **Orange: 247,150,70**
- COMMON **yellow: 241,224,151**

8.4.2 HEX Colors

- Blue: 107,203,236 → #6BCBEC
- - Green: 155,187,89 → #9BBB59
- - Dark Blue: 79,129,189 → #4F81BD

- - Orange: 247,150,70 → #F79646
- - Dark Orange: 233,191,149 → #E9BF95
- - Violet: 234,198,234 → #EAC6EA
- - Red: 222,103,103 → #DE6767
- - Light Blue: 240,255,255 → #F0FFFF
- - Green (another shade): 156,204,84 → #9CCC54
- - Yellow: 241,224,151 → #F1E097